

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
import plotly.express as px
```

Load Dataset

```
df = pd.read_csv (r"C:\Users\A0808\OneDrive\Desktop\Q-Comm\
quick_commerce_data_raw.csv")
```

df

	Order_ID	Company	City	Customer_Age
Order_Value \				
0	1000001	Swiggy Instamart	Noida	46
702.33750				
1	1000002	Flipkart Minutes	Amritsar	56
1007.30000				
2	1000003	Flipkart Minutes	Mumbai	18
1211.66000				
3	1000004	Swiggy Instamart	Delhi	23
1179.05925				
4	1000005	Dunzo	Mumbai	44
586.02550				
...	...	...	...	...
.				..
999995	1999996	Big Basket	Mumbai	48
72.33000				
999996	1999997	Swiggy Instamart	Mumbai	33
1509.66300				
999997	1999998	Jio Mart	Noida	29
637.32300				
999998	1999999	Dunzo	Pune	42
1103.12100				
999999	2000000	Amazon Now	Chennai	19
758.58860				
	Delivery_Time_Min	Distance_Km	Items_Count	
Product_Category \				
0	19.182	11.97	12.0	
Dairy				
1	19.644	12.74	10.0	
Snacks				
2	16.910	4.85	NaN	Personal
Care				
3	5.864	6.44	2.0	
Dairy				
4	12.470	2.45	13.0	

```
Household
...
999995      17.806      3.01      19.0  Fruits &
Vegetables
999996      18.306      10.51      9.0
Beverages
999997      17.590      2.65      6.0
Groceries
999998      12.656      7.76      12.0
Snacks
999999      17.670      9.45      13.0
Snacks
```

```

      Payment_Method  Customer_Rating  Discount_Applied \
0           Wallet           2.1           1
1  Cash on Delivery           2.3           0
2  Cash on Delivery           3.3           0
3   Credit Card           5.0           1
4           Wallet           3.7           0
...
999995      Wallet           4.2           0
999996  Debit Card           4.1           1
999997  Cash on Delivery           3.4           0
999998      Wallet           4.0           1
999999  Credit Card           4.1           0
```

```

      Delivery_Partner_Rating
0           3.2
1           3.2
2           3.8
3           5.0
4           4.8
...
999995      2.8
999996      3.5
999997      4.6
999998      4.5
999999      4.8
```

```
[1000000 rows x 13 columns]
```

#Data Structure

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 13 columns):
```

```
#    Column                                Non-Null Count  Dtype
```

```

---
0  Order_ID          1000000 non-null int64
1  Company           1000000 non-null object
2  City              948000 non-null object
3  Customer_Age      1000000 non-null int64
4  Order_Value       1000000 non-null float64
5  Delivery_Time_Min 1000000 non-null float64
6  Distance_Km        1000000 non-null float64
7  Items_Count       965000 non-null float64
8  Product_Category  1000000 non-null object
9  Payment_Method     1000000 non-null object
10 Customer_Rating    953000 non-null float64
11 Discount_Applied   1000000 non-null int64
12 Delivery_Partner_Rating 895863 non-null float64
dtypes: float64(6), int64(3), object(4)
memory usage: 99.2+ MB

```

#Summary Statistics

```
df.describe()
```

	Order_ID	Customer_Age	Order_Value	Delivery_Time_Min
\count	1.000000e+06	1000000.000000	1000000.000000	1000000.000000
mean	1.500000e+06	38.470387	571.643584	16.452412
std	2.886753e+05	12.118520	382.784520	6.257172
min	1.000001e+06	18.000000	50.000000	5.000000
25%	1.250001e+06	28.000000	287.840150	12.368000
50%	1.500000e+06	38.000000	531.324600	16.342000
75%	1.750000e+06	49.000000	796.182500	20.196000
max	2.000000e+06	59.000000	13877.325000	40.000000

	Distance_Km	Items_Count	Customer_Rating
Discount_Applied \count	1000000.000000	965000.000000	953000.000000
1000000.000000			
mean	7.750231	9.998317	3.042072
0.400883			
std	4.186665	5.478725	1.186872
0.490078			
min	0.500000	1.000000	1.000000
0.000000			
25%	4.130000	5.000000	2.000000

```

0.000000
50%          7.740000      10.000000      3.000000
0.000000
75%          11.380000      15.000000      4.000000
1.000000
max           15.000000      19.000000      5.000000
1.000000

```

```

      Delivery_Partner_Rating
count      895863.000000
mean              3.749257
std              0.722121
min              2.500000
25%              3.100000
50%              3.700000
75%              4.400000
max              5.000000

```

Check Missing Values NULL & NA

```
df.isna().sum()
```

```

Order_ID          0
Company           0
City             52000
Customer_Age      0
Order_Value       0
Delivery_Time_Min 0
Distance_Km       0
Items_Count      35000
Product_Category  0
Payment_Method    0
Customer_Rating   47000
Discount_Applied  0
Delivery_Partner_Rating  104137
dtype: int64

```

To show the records which contains atleast ANY one or ALL null values.

```
df[df.isna().any(axis=1)]
```

```

      Order_ID      Company      City  Customer_Age
Order_Value \
2      1000003  Flipkart Minutes      Mumbai      18
1211.66000
6      1000007      Blinkit  Bengluru      37
516.86480
17     1000018      Zepto    Delhi      23
1041.42975

```

21	1000022	Blinkit	NaN	41
716.89860				
22	1000023	Jio Mart	Delhi	48
50.00000				
...	...	...	...	...
..				
999966	1999967	Dunzo	NaN	39
783.89250				
999975	1999976	Amazon Now	Bengluru	34
515.89380				
999981	1999982	Amazon Now	NaN	33
939.34470				
999983	1999984	Swiggy Instamart	Hyderabad	42
50.00000				
999990	1999991	Amazon Now	Delhi	39
781.65420				

	Delivery_Time_Min	Distance_Km	Items_Count	
Product_Category \				
2	16.910	4.85	NaN	Personal
Care				
6	18.476	12.46	19.0	
Snacks				
17	5.000	14.74	18.0	
Snacks				
21	17.804	11.34	7.0	
Beverages				
22	8.498	0.83	3.0	
Snacks				
...	...	...	...	...
..				

999966	18.820	14.70	NaN	
Household				
999975	21.294	10.49	10.0	Fruits &
Vegetables				
999981	21.080	11.80	18.0	
Dairy				
999983	13.386	2.31	2.0	
Groceries				
999990	7.158	6.93	NaN	Personal
Care				

	Payment_Method	Customer_Rating	Discount_Applied	\
2	Cash on Delivery	3.3	0	
6	Debit Card	2.8	0	
17	Wallet	NaN	1	
21	Credit Card	2.7	1	
22	Cash on Delivery	3.1	0	
...	...	...	...	...

999966	Wallet	3.1	1
999975	Wallet	NaN	1
999981	Wallet	3.2	1
999983	Wallet	NaN	0
999990	Wallet	4.0	0

Delivery_Partner_Rating	
2	3.8
6	NaN
17	NaN
21	3.6
22	NaN
...	...
999966	4.9
999975	3.0
999981	4.4
999983	3.2
999990	3.9

[219008 rows x 13 columns]

To show the records which contains ALL null values.

```
df[df.isnull().all(axis=1)]
```

Empty DataFrame

Columns: [Order_ID, Company, City, Customer_Age, Order_Value, Delivery_Time_Min, Distance_Km, Items_Count, Product_Category, Payment_Method, Customer_Rating, Discount_Applied, Delivery_Partner_Rating]
Index: []

*# Drop those rows that have missing values in those column which are of STRING/CATEGORY Format
#eg; City is a string format hence we can't use mean/mode method to replace those missing values*

```
df= df.dropna(subset= ['City'] )
```

df

	Order_ID	Company	City	Customer_Age
Order_Value \				
0	1000001	Swiggy Instamart	Noida	46
702.33750				
1	1000002	Flipkart Minutes	Amritsar	56
1007.30000				
2	1000003	Flipkart Minutes	Mumbai	18
1211.66000				
3	1000004	Swiggy Instamart	Delhi	23
1179.05925				

4	1000005	Dunzo	Mumbai	44	
586.02550					
...	...	...	...	...	..
.					
999995	1999996	Big Basket	Mumbai	48	
72.33000					
999996	1999997	Swiggy Instamart	Mumbai	33	
1509.66300					
999997	1999998	Jio Mart	Noida	29	
637.32300					
999998	1999999	Dunzo	Pune	42	
1103.12100					
999999	2000000	Amazon Now	Chennai	19	
758.58860					
	Delivery_Time_Min	Distance_Km	Items_Count		
Product_Category \					
0	19.182	11.97	12.0		
Dairy					
1	19.644	12.74	10.0		
Snacks					
2	16.910	4.85	NaN	Personal	
Care					
3	5.864	6.44	2.0		
Dairy					
4	12.470	2.45	13.0		
Household					
...	...	...	...		.
..					
999995	17.806	3.01	19.0	Fruits &	
Vegetables					
999996	18.306	10.51	9.0		
Beverages					
999997	17.590	2.65	6.0		
Groceries					
999998	12.656	7.76	12.0		
Snacks					
999999	17.670	9.45	13.0		
Snacks					
	Payment_Method	Customer_Rating	Discount_Applied	\	
0	Wallet	2.1	1		
1	Cash on Delivery	2.3	0		
2	Cash on Delivery	3.3	0		
3	Credit Card	5.0	1		
4	Wallet	3.7	0		
...	...	...	...		
999995	Wallet	4.2	0		
999996	Debit Card	4.1	1		

999997	Cash on Delivery	3.4	0
999998	Wallet	4.0	1
999999	Credit Card	4.1	0

Delivery_Partner_Rating	
0	3.2
1	3.2
2	3.8
3	5.0
4	4.8
...	...
999995	2.8
999996	3.5
999997	4.6
999998	4.5
999999	4.8

[948000 rows x 13 columns]

Check Missing Values NULL & NA

df.isna().sum()

Order_ID	0
Company	0
City	0
Customer_Age	0
Order_Value	0
Delivery_Time_Min	0
Distance_Km	0
Items_Count	33228
Product_Category	0
Payment_Method	0
Customer_Rating	44575
Discount_Applied	0
Delivery_Partner_Rating	98694

dtype: int64

df.shape

(948000, 13)

Check Mode

```
mode = df['Items_Count'].mode()
median = df['Items_Count'].median()
mean = df['Items_Count'].mean()
```

mean

np.float64(9.997801638003788)

```
median
```

```
10.0
```

```
mode
```

```
0    19.0
```

```
Name: Items_Count, dtype: float64
```

```
#Fill the missing value with MODE
```

```
#[0] = index value 0
```

```
#inplace = to make the changes permanent
```

```
df['Items_Count'].fillna(mode[0], inplace = True)
```

```
C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\1300746656.py:5:
```

```
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
```

```
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
```

```
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.
```

```
df['Items_Count'].fillna(mode[0], inplace = True)
```

```
C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\1300746656.py:5:
```

```
SettingWithCopyWarning:
```

```
A value is trying to be set on a copy of a slice from a DataFrame
```

```
See the caveats in the documentation:
```

```
https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
```

```
df['Items_Count'].fillna(mode[0], inplace = True)
```

```
# Check Missing Values NULL & NA
```

```
df.isna().sum()
```

Order_ID	0
Company	0
City	0
Customer_Age	0
Order_Value	0
Delivery_Time_Min	0
Distance_Km	0
Items_Count	0
Product_Category	0

```
Payment_Method          0
Customer_Rating         44575
Discount_Applied        0
Delivery_Partner_Rating 98694
dtype: int64
```

```
# Fill Null values with Group-wise Mean/Median (To fill blank
customer_rating column)
```

```
df['Customer_Rating'] = df.groupby("Company")
["Customer_Rating"].transform(lambda x:x.fillna(x.mean() ))
```

```
C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\1778484687.py:3:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['Customer_Rating'] = df.groupby("Company")
["Customer_Rating"].transform(lambda x:x.fillna(x.mean() ))
```

```
# Check missing values
```

```
df.isna().sum()
```

```
Order_ID          0
Company           0
City              0
Customer_Age      0
Order_Value       0
Delivery_Time_Min 0
Distance_Km       0
Items_Count       0
Product_Category  0
Payment_Method    0
Customer_Rating   0
Discount_Applied  0
Delivery_Partner_Rating 98694
dtype: int64
```

```
# Fill Null values with Group-wise Mean/Median (To fill blank
Delivery_Partner_Rating column)
```

```
df['Delivery_Partner_Rating'] = df.groupby("Delivery_Time_Min")
['Delivery_Partner_Rating'].transform(lambda y:y.fillna (y.mean()))
```

```
C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\100600045.py:3:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
```

Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['Delivery_Partner_Rating'] = df.groupby("Delivery_Time_Min")
['Delivery_Partner_Rating'].transform(lambda y:y.fillna (y.mean()))
```

Check missing values

```
df.isna().sum()
```

Order_ID	0
Company	0
City	0
Customer_Age	0
Order_Value	0
Delivery_Time_Min	0
Distance_Km	0
Items_Count	0
Product_Category	0
Payment_Method	0
Customer_Rating	0
Discount_Applied	0
Delivery_Partner_Rating	76
dtype:	int64

```
df['Delivery_Partner_Rating'] = df['Delivery_Partner_Rating'].fillna
(df['Delivery_Partner_Rating'].mean())
```

C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\1264346101.py:1:

SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['Delivery_Partner_Rating'] = df['Delivery_Partner_Rating'].fillna
(df['Delivery_Partner_Rating'].mean())
```

Check missing values

```
df.isna().sum()
```

Order_ID	0
Company	0
City	0
Customer_Age	0
Order_Value	0
Delivery_Time_Min	0

```

Distance_Km          0
Items_Count          0
Product_Category     0
Payment_Method       0
Customer_Rating      0
Discount_Applied     0
Delivery_Partner_Rating 0
dtype: int64

df.info()

<class 'pandas.core.frame.DataFrame'>
Index: 948000 entries, 0 to 999999
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Order_ID              948000 non-null  int64
1   Company               948000 non-null  object
2   City                  948000 non-null  object
3   Customer_Age          948000 non-null  int64
4   Order_Value           948000 non-null  float64
5   Delivery_Time_Min     948000 non-null  float64
6   Distance_Km           948000 non-null  float64
7   Items_Count           948000 non-null  float64
8   Product_Category      948000 non-null  object
9   Payment_Method        948000 non-null  object
10  Customer_Rating       948000 non-null  float64
11  Discount_Applied      948000 non-null  int64
12  Delivery_Partner_Rating 948000 non-null  float64
dtypes: float64(6), int64(3), object(4)
memory usage: 101.3+ MB

```

Removing Outliers

```

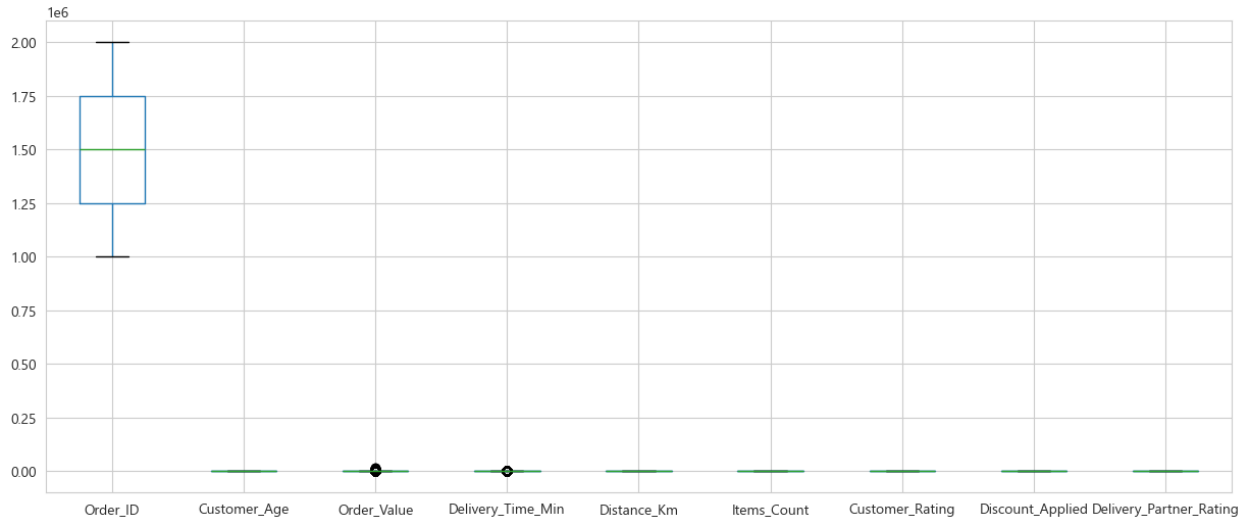
# Box Plot

plt.figure(figsize= (15,6))

df.boxplot()

<Axes: >

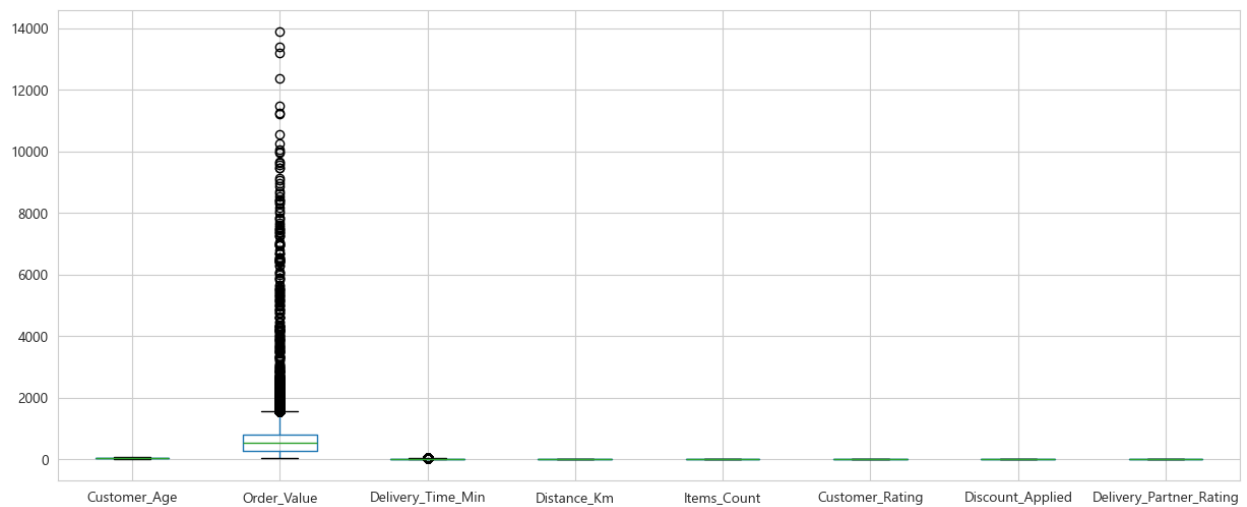
```



Remove Order_ID column bcoz we have all unique id upto 1 million, therefore we will drop that particular column

```
plt.figure(figsize= (15,6))
df.drop(columns = ["Order_ID"]).boxplot()
```

<Axes: >



```
# Remove outlier
# Method 1) Percentile (Capping) method

#         df['Order_Value'] = df['Order_Value'].clip(upper = 2500)
# Method 2) Filtering
df = df[df['Order_Value'] <= 2500]
```

df

Order_ID		Company	City	Customer_Age
Order_Value \				
0	1000001	Swiggy Instamart	Noida	46
702.33750				
1	1000002	Flipkart Minutes	Amritsar	56
1007.30000				
2	1000003	Flipkart Minutes	Mumbai	18
1211.66000				
3	1000004	Swiggy Instamart	Delhi	23
1179.05925				
4	1000005	Dunzo	Mumbai	44
586.02550				
...	...	...	...	...
.				
999995	1999996	Big Basket	Mumbai	48
72.33000				
999996	1999997	Swiggy Instamart	Mumbai	33
1509.66300				
999997	1999998	Jio Mart	Noida	29
637.32300				
999998	1999999	Dunzo	Pune	42
1103.12100				
999999	2000000	Amazon Now	Chennai	19
758.58860				
Delivery_Time_Min		Distance_Km	Items_Count	
Product_Category \				
0	19.182	11.97	12.0	
Dairy				
1	19.644	12.74	10.0	
Snacks				
2	16.910	4.85	19.0	Personal
Care				
3	5.864	6.44	2.0	
Dairy				
4	12.470	2.45	13.0	
Household				
...	...	...	...	.
..				
999995	17.806	3.01	19.0	Fruits &
Vegetables				
999996	18.306	10.51	9.0	
Beverages				
999997	17.590	2.65	6.0	
Groceries				
999998	12.656	7.76	12.0	
Snacks				
999999	17.670	9.45	13.0	

Snacks

	Payment_Method	Customer_Rating	Discount_Applied	\
0	Wallet	2.1	1	
1	Cash on Delivery	2.3	0	
2	Cash on Delivery	3.3	0	
3	Credit Card	5.0	1	
4	Wallet	3.7	0	
...	...	...	...	
999995	Wallet	4.2	0	
999996	Debit Card	4.1	1	
999997	Cash on Delivery	3.4	0	
999998	Wallet	4.0	1	
999999	Credit Card	4.1	0	

	Delivery_Partner_Rating
0	3.2
1	3.2
2	3.8
3	5.0
4	4.8
...	...
999995	2.8
999996	3.5
999997	4.6
999998	4.5
999999	4.8

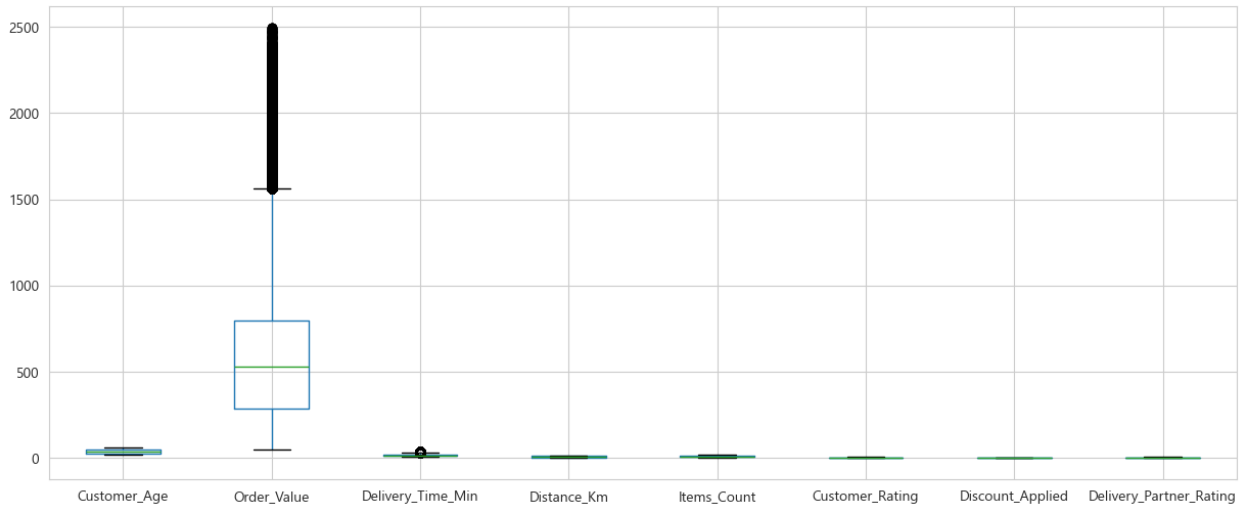
[947752 rows x 13 columns]

Remove Order_ID column bcoz we have all unique id upto 1 million, therefore we will drop that particular column

```
plt.figure(figsize= (15,6))
```

```
df.drop(columns = ["Order_ID"]).boxplot()
```

```
plt.show()
```

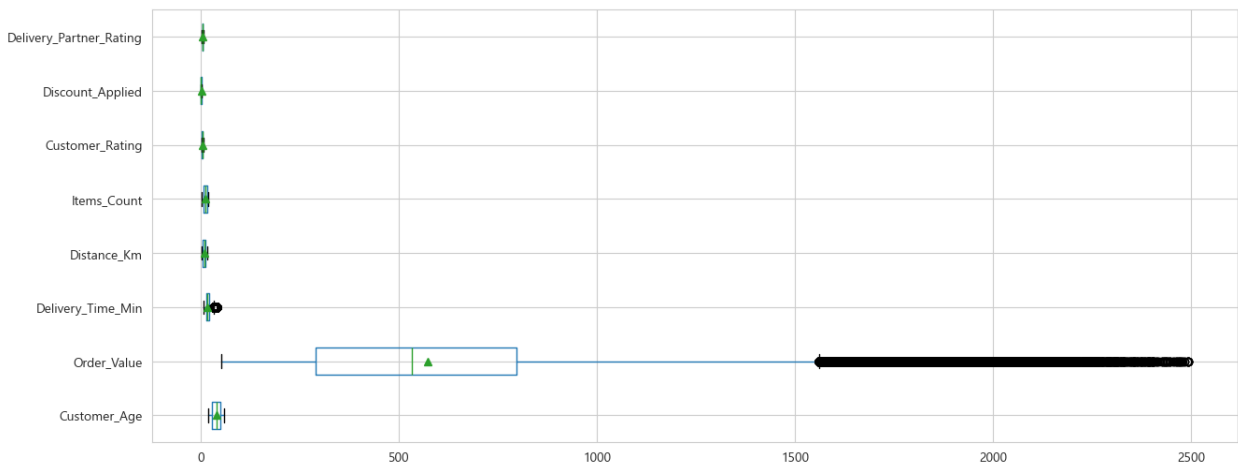


Remove Order_ID column bcoz we have all unique id upto 1 million, therefore we will drop that particular column (HORIZONTAL BOX PLOT)

```
plt.figure(figsize= (15,6))
```

```
df.drop(columns = ["Order_ID"]).boxplot(vert = False , showmeans = True )
```

<Axes: >



Change the Data Type of the Columns

#Dataset Structure

```
df .info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 947752 entries, 0 to 999999
```

```
Data columns (total 13 columns):
```

```
#    Column                                Non-Null Count  Dtype
```

```

---
0  Order_ID          947752 non-null  int64
1  Company          947752 non-null  object
2  City             947752 non-null  object
3  Customer_Age     947752 non-null  int64
4  Order_Value      947752 non-null  float64
5  Delivery_Time_Min 947752 non-null  float64
6  Distance_Km      947752 non-null  float64
7  Items_Count      947752 non-null  float64
8  Product_Category 947752 non-null  object
9  Payment_Method    947752 non-null  object
10 Customer_Rating   947752 non-null  float64
11 Discount_Applied  947752 non-null  int64
12 Delivery_Partner_Rating 947752 non-null float64
dtypes: float64(6), int64(3), object(4)
memory usage: 101.2+ MB

```

Change the data type of Order ID

```
df['Order_ID'] = df['Order_ID'].astype(str)
```

C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\611162501.py:3:

SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['Order_ID'] = df['Order_ID'].astype(str)
```

Round-off the float values and change data type of Order_Value to int

```

df['Order_Value'] = np.round(df['Order_Value'])
df['Delivery_Time_Min'] = np.round(df['Delivery_Time_Min'])
df['Distance_Km'] = np.round(df['Distance_Km'], 1)
df['Customer_Rating'] = np.round(df['Customer_Rating'])
df['Delivery_Partner_Rating'] = np.round(df['Delivery_Partner_Rating'])

```

C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\3431222525.py:3:

SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['Order_Value'] = np.round(df['Order_Value'])
```

C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\3431222525.py:4:

SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['Delivery_Time_Min']= np.round(df['Delivery_Time_Min'])
C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\3431222525.py:5:
SettingWithCopyWarning:
```

A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['Distance_Km']= np.round(df['Distance_Km'], 1)
C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\3431222525.py:6:
SettingWithCopyWarning:
```

A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['Customer_Rating']= np.round(df['Customer_Rating'])
C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\3431222525.py:7:
SettingWithCopyWarning:
```

A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['Delivery_Partner_Rating']=
np.round(df['Delivery_Partner_Rating'])
```

```
df.head()
```

	Order_ID	Company	City	Customer_Age	Order_Value	\
0	1000001	Swiggy Instamart	Noida	46	702.0	
1	1000002	Flipkart Minutes	Amritsar	56	1007.0	
2	1000003	Flipkart Minutes	Mumbai	18	1212.0	
3	1000004	Swiggy Instamart	Delhi	23	1179.0	
4	1000005	Dunzo	Mumbai	44	586.0	

	Delivery_Time_Min	Distance_Km	Items_Count	Product_Category	\
0	19.0	12.0	12.0	Dairy	
1	20.0	12.7	10.0	Snacks	
2	17.0	4.8	19.0	Personal Care	
3	6.0	6.4	2.0	Dairy	

	12.0	2.4	13.0	Household
4	Payment_Method	Customer_Rating	Discount_Applied	\
0	Wallet	2.0		1
1	Cash on Delivery	2.0		0
2	Cash on Delivery	3.0		0
3	Credit Card	5.0		1
4	Wallet	4.0		0

	Delivery_Partner_Rating
0	3.0
1	3.0
2	4.0
3	5.0
4	5.0

Change the data-type of float columns to integer

```
df['Order_Value'] = df['Order_Value'].astype(int)
df['Delivery_Time_Min'] = df['Delivery_Time_Min'].astype(int)
df['Items_Count'] = df['Items_Count'].astype(int)
df['Customer_Rating'] = df['Customer_Rating'].astype(int)
df['Delivery_Partner_Rating'] = df['Delivery_Partner_Rating'].astype(int)
```

C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\724973111.py:3:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['Order_Value'] = df['Order_Value'].astype(int)
```

C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\724973111.py:4:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['Delivery_Time_Min'] = df['Delivery_Time_Min'].astype(int)
```

C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\724973111.py:5:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
returning-a-view-versus-a-copy
df['Items_Count']= df['Items_Count']. astype(int)
C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\724973111.py:6:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['Customer_Rating']= df['Customer_Rating']. astype(int)
C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\724973111.py:7:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['Delivery_Partner_Rating']= df['Delivery_Partner_Rating'].
astype(int)
```

#Dataset Structure

```
df .info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 947752 entries, 0 to 999999
```

```
Data columns (total 13 columns):
```

#	Column	Non-Null Count	Dtype
0	Order_ID	947752 non-null	object
1	Company	947752 non-null	object
2	City	947752 non-null	object
3	Customer_Age	947752 non-null	int64
4	Order_Value	947752 non-null	int64
5	Delivery_Time_Min	947752 non-null	int64
6	Distance_Km	947752 non-null	float64
7	Items_Count	947752 non-null	int64
8	Product_Category	947752 non-null	object
9	Payment_Method	947752 non-null	object
10	Customer_Rating	947752 non-null	int64
11	Discount_Applied	947752 non-null	int64
12	Delivery_Partner_Rating	947752 non-null	int64

```
dtypes: float64(1), int64(7), object(5)
```

```
memory usage: 101.2+ MB
```

Save Cleaned & Modified dataset

```
New_Path=
r"C:\Users\A0808\OneDrive\Desktop\Q-Comm\quick_commerce_data_modified_
cleaned.csv"

df.to_csv(New_Path, index= False)

#index = False : to remove the index
```

Load Cleaned & Modified dataset

```
df = pd.read_csv(r"C:\Users\A0808\OneDrive\Desktop\Q-Comm\
quick_commerce_data_modified_cleaned.csv")
```

df

	Order_ID	Company	City	Customer_Age
Order_Value \				
0	1000001	Swiggy Instamart	Noida	46
702				
1	1000002	Flipkart Minutes	Amritsar	56
1007				
2	1000003	Flipkart Minutes	Mumbai	18
1212				
3	1000004	Swiggy Instamart	Delhi	23
1179				
4	1000005	Dunzo	Mumbai	44
586				
...	...	...	...	...
.				..
947747	1999996	Big Basket	Mumbai	48
72				
947748	1999997	Swiggy Instamart	Mumbai	33
1510				
947749	1999998	Jio Mart	Noida	29
637				
947750	1999999	Dunzo	Pune	42
1103				
947751	2000000	Amazon Now	Chennai	19
759				
	Delivery_Time_Min	Distance_Km	Items_Count	
Product_Category \				
0	19	12.0	12	
Dairy				
1	20	12.7	10	
Snacks				
2	17	4.8	19	Personal
Care				

3	6	6.4	2	
Dairy				
4	12	2.4	13	
Household				
...	...	...	...	.
..				
947747	18	3.0	19	Fruits &
Vegetables				
947748	18	10.5	9	
Beverages				
947749	18	2.6	6	
Groceries				
947750	13	7.8	12	
Snacks				
947751	18	9.4	13	
Snacks				

	Payment_Method	Customer_Rating	Discount_Applied	\
0	Wallet	2	1	
1	Cash on Delivery	2	0	
2	Cash on Delivery	3	0	
3	Credit Card	5	1	
4	Wallet	4	0	
...	...	...	...	
947747	Wallet	4	0	
947748	Debit Card	4	1	
947749	Cash on Delivery	3	0	
947750	Wallet	4	1	
947751	Credit Card	4	0	

	Delivery_Partner_Rating
0	3
1	3
2	4
3	5
4	5
...	...
947747	3
947748	4
947749	5
947750	4
947751	5

[947752 rows x 13 columns]

Probable Questions

Q1) which quick commerce platform has the highest total revenue?

```
company_revenue = df.groupby("Company")["Order_Value"].sum()
```

```
## OR Else company_revenue = df.groupby("Company")  
["Order_Value"].sum().sort_values()
```

```
company_revenue
```

```
Company  
Amazon Now      65832504  
Big Basket      67884762  
Blinkit         72499575  
Dunzo           64007579  
Flipkart Minutes 66998289  
Jio Mart        57112860  
Swiggy Instamart 76407756  
Zepto           70339672  
Name: Order_Value, dtype: int64
```

```
company_revenue.sort_values()
```

```
Company  
Jio Mart        57112860  
Dunzo           64007579  
Amazon Now      65832504  
Flipkart Minutes 66998289  
Big Basket      67884762  
Zepto           70339672  
Blinkit         72499575  
Swiggy Instamart 76407756  
Name: Order_Value, dtype: int64
```

```
# Bar and line chart
```

```
plt.figure(figsize =(8,5))
```

```
# Find max and min value
```

```
max_value = company_revenue.max()
```

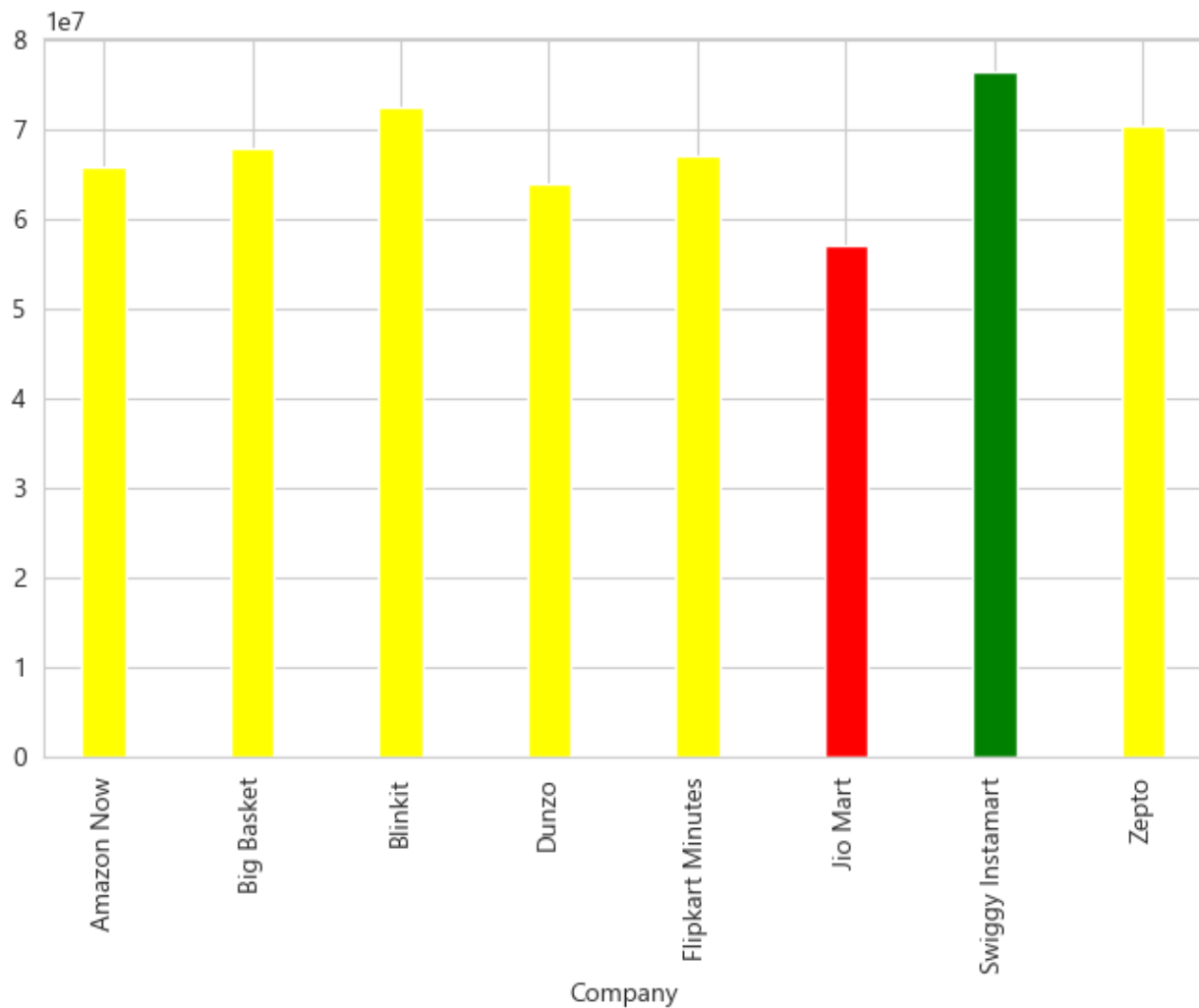
```
min_value =company_revenue.min()
```

```
# setting colours
```

```
colors = [ 'green' if val ==max_value else  
           'red' if val == min_value else  
           'yellow'  
           for val in company_revenue]
```

```
# Draw a bar chart
```

```
company_revenue.plot(kind = 'bar', color = colors , width =0.3 )  
plt.show()
```



```
# Bar and line chart
```

```
plt.figure(figsize =(10,5))
```

```
# Find max and min value
```

```
max_value = company_revenue.max()
```

```
min_value =company_revenue.min()
```

```
# setting colours
```

```
colors = [ 'green'if val ==max_value else
```

```

        'red' if val == min_value else
        'yellow'
        for val in company_revenue]

# Draw a bar chart

company_revenue.plot(kind = 'bar', color = colors , width = 0.3 )
company_revenue.plot(kind = 'line', color = 'black' , marker = '*')

# Title & x,y label
plt.xlabel ( "company", fontsize = 12)
plt.ylabel ( "Total Revenue", fontsize = 12)
plt.title("Total Revenue by Company" , fontsize = 14)
plt.xticks(rotation=45, ha='right' )

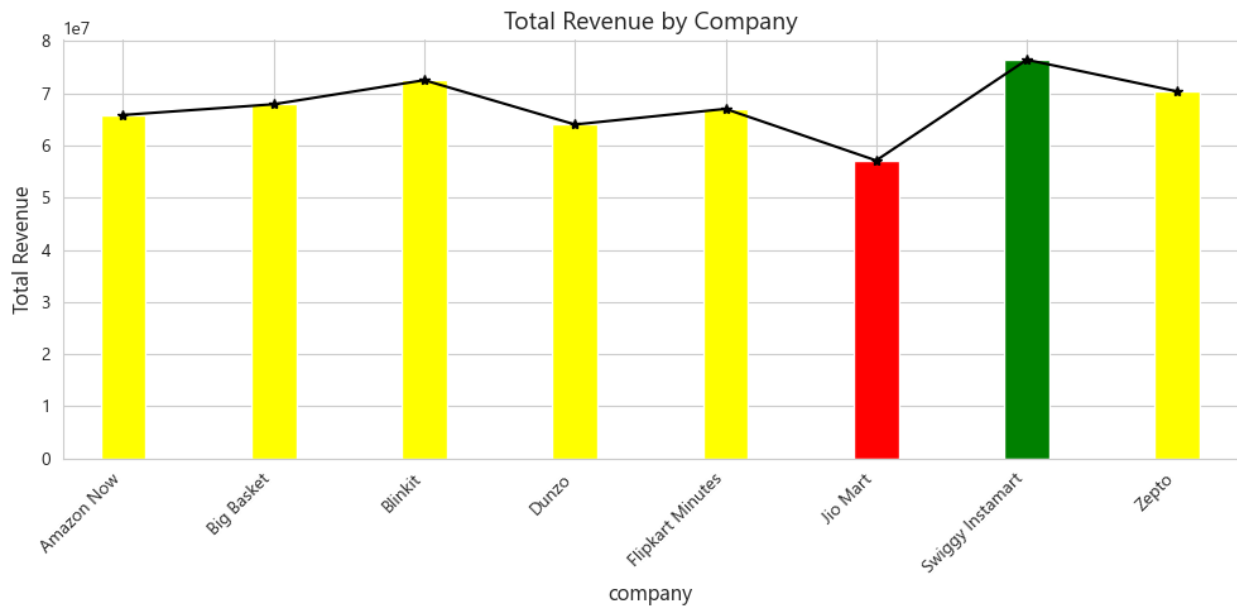
# Removing the top and the right border
ax = plt.gca () # gca = get current axes
ax.spines['top'].set_visible(False)

ax.spines['right'].set_visible(False)

plt.tight_layout()

plt.show()

```



Q2) Which platform has the highest average order value (AOV) ?

```
df.head()
```

	Order_ID	Company	City	Customer_Age	Order_Value	\
0	1000001	Swiggy Instamart	Noida	46	702	
1	1000002	Flipkart Minutes	Amritsar	56	1007	
2	1000003	Flipkart Minutes	Mumbai	18	1212	
3	1000004	Swiggy Instamart	Delhi	23	1179	
4	1000005	Dunzo	Mumbai	44	586	

	Delivery_Time_Min	Distance_Km	Items_Count	Product_Category	\
0	19	12.0	12	Dairy	
1	20	12.7	10	Snacks	
2	17	4.8	19	Personal Care	
3	6	6.4	2	Dairy	
4	12	2.4	13	Household	

	Payment_Method	Customer_Rating	Discount_Applied	\
0	Wallet	2	1	
1	Cash on Delivery	2	0	
2	Cash on Delivery	3	0	
3	Credit Card	5	1	
4	Wallet	4	0	

	Delivery_Partner_Rating
0	3
1	3
2	4
3	5
4	5

```
# Group data - average order value of caomany (mean)
```

```
AOV = df.groupby("Company")["Order_Value"].mean()
```

```
AOV.sort_values()
```

Company	
Jio Mart	482.914592
Dunzo	540.317053
Amazon Now	557.510429
Flipkart Minutes	563.176472
Big Basket	575.065542
Zepto	593.258314
Blinkit	609.819198
Swiggy Instamart	644.927250

Name: Order_Value, dtype: float64

```

# Stem chart
plt.figure(figsize= (10,4))

plt.stem(A0V.index, A0V.values ) # inside the bracket it is x and y
values

# Title & x,y label
plt.xlabel ( "company", fontsize = 12)
plt.ylabel ( "Average_order_value(INR)", fontsize = 12)
plt.title("TAverage_order_value by Company" , fontsize = 14)
plt.xticks(rotation=30, ha='right' )

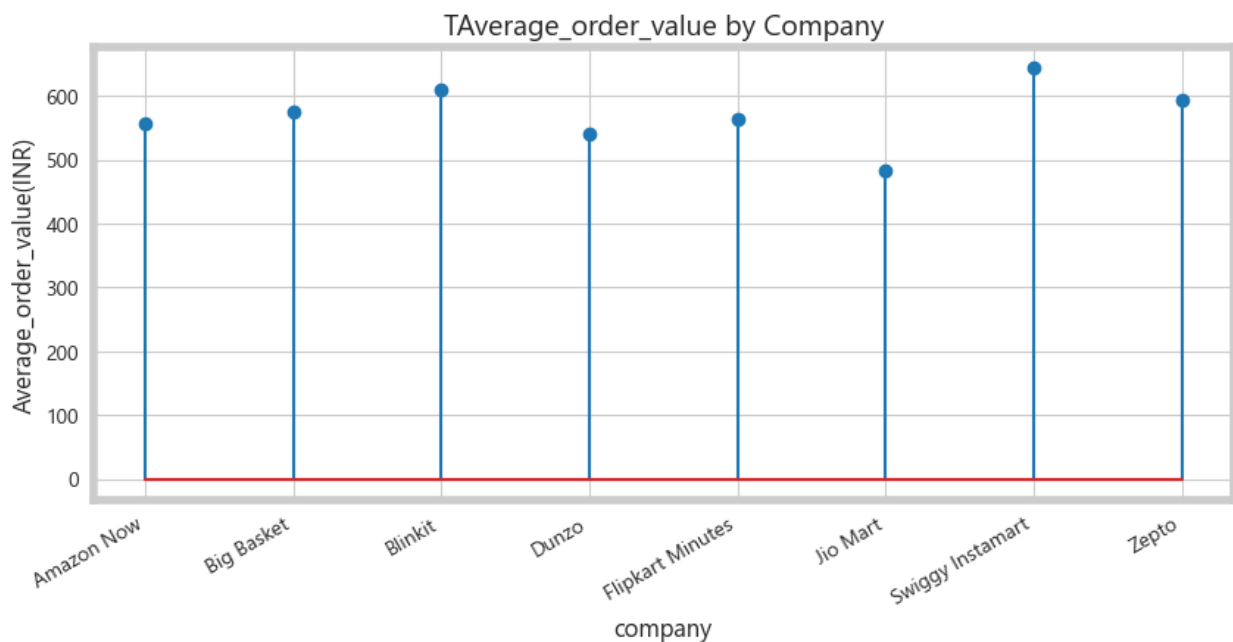
# Highlight all four borders

ax = plt.gca () # gca = get current axis

ax.spines['top'].set_linewidth(4)
ax.spines['right'].set_linewidth(4)
ax.spines['left'].set_linewidth(4)
ax.spines['bottom'].set_linewidth(4)

plt.show()

```



Q3) How does customer rating vary across platforms?

```
df.head()
```

	Order_ID	Company	City	Customer_Age	Order_Value	\
0	1000001	Swiggy Instamart	Noida	46	702	

1	1000002	Flipkart Minutes	Amritsar	56	1007
2	1000003	Flipkart Minutes	Mumbai	18	1212
3	1000004	Swiggy Instamart	Delhi	23	1179
4	1000005	Dunzo	Mumbai	44	586

	Delivery_Time_Min	Distance_Km	Items_Count	Product_Category	\
0	19	12.0	12	Dairy	
1	20	12.7	10	Snacks	
2	17	4.8	19	Personal Care	
3	6	6.4	2	Dairy	
4	12	2.4	13	Household	

	Payment_Method	Customer_Rating	Discount_Applied	\
0	Wallet	2	1	
1	Cash on Delivery	2	0	
2	Cash on Delivery	3	0	
3	Credit Card	5	1	
4	Wallet	4	0	

	Delivery_Partner_Rating
0	3
1	3
2	4
3	5
4	5

```
median = df.groupby('Company')['Customer_Rating'].median()
```

```
median
```

```
Company
Amazon Now      3.0
Big Basket      3.0
Blinkit         4.0
Dunzo           2.0
Flipkart Minutes 3.0
Jio Mart        3.0
Swiggy Instamart 3.0
Zepto           3.0
Name: Customer_Rating, dtype: float64
```

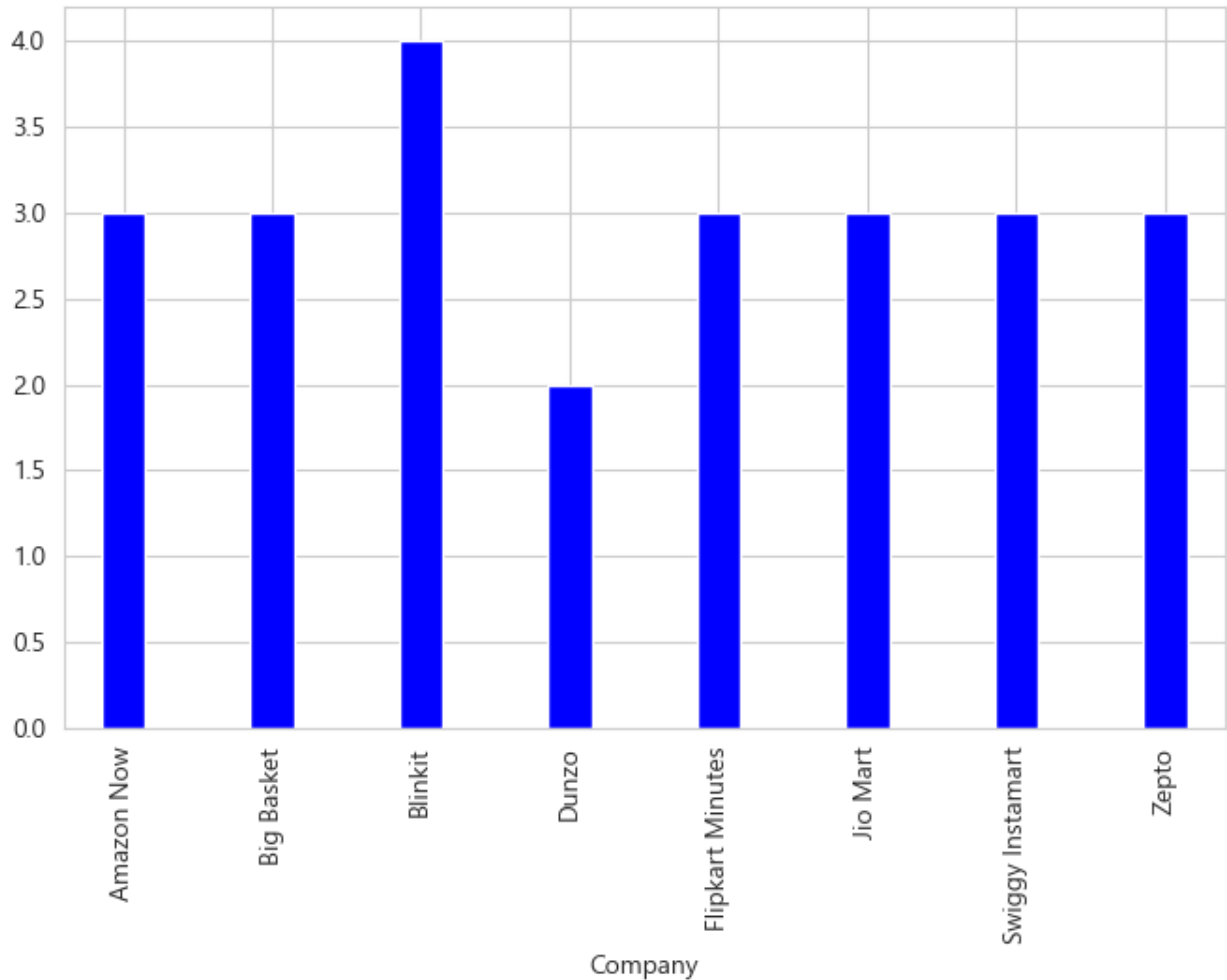
```
# mode = ( df.groupby('Company')
#         ['Customer_Rating']
#         .agg(pd.Series.mode)
#         .reset_index() )
```

```
#mode
```

```
# Bar Chart
```

```
plt.figure(figsize = (8,5))
```

```
median.plot(kind = 'bar', color = 'blue' , width =0.3 )  
<Axes: xlabel='Company'>
```



```
## Box Plot
```

```
df.boxplot (column = 'Customer_Rating' , by = 'Company', showmeans =  
True )
```

```
#Labeling
```

```
plt.title ("customer rating distribution across Company")  
plt.xlabel("Company")  
plt.ylabel("Customer_Rating")  
plt.xticks(rotation = 30, ha = 'right')
```

```
# Removing the default super title
```

```
plt.suptitle("")
```

```
plt.tight_layout()      # Auto adjust spacing
plt.show()
```



```
# Count of Ratings
grouped_rating = df.groupby(['Company',
                              'Customer_Rating']).size().unstack(fill_value = 0)
```

grouped_rating

Customer_Rating	1	2	3	4	5
Company					
Amazon Now	15977	30351	30866	29697	11192
Big Basket	10143	30339	31020	30908	15637
Blinkit	1060	25553	25561	36708	30005
Dunzo	29818	36280	25377	25818	1170
Flipkart Minutes	12549	30786	31225	30985	13420
Jio Mart	18513	30515	31083	30089	8067
Swiggy Instamart	5000	30278	31039	30931	21227
Zepto	7618	29763	31367	30559	19258

```
# Bar chart for grouped rating

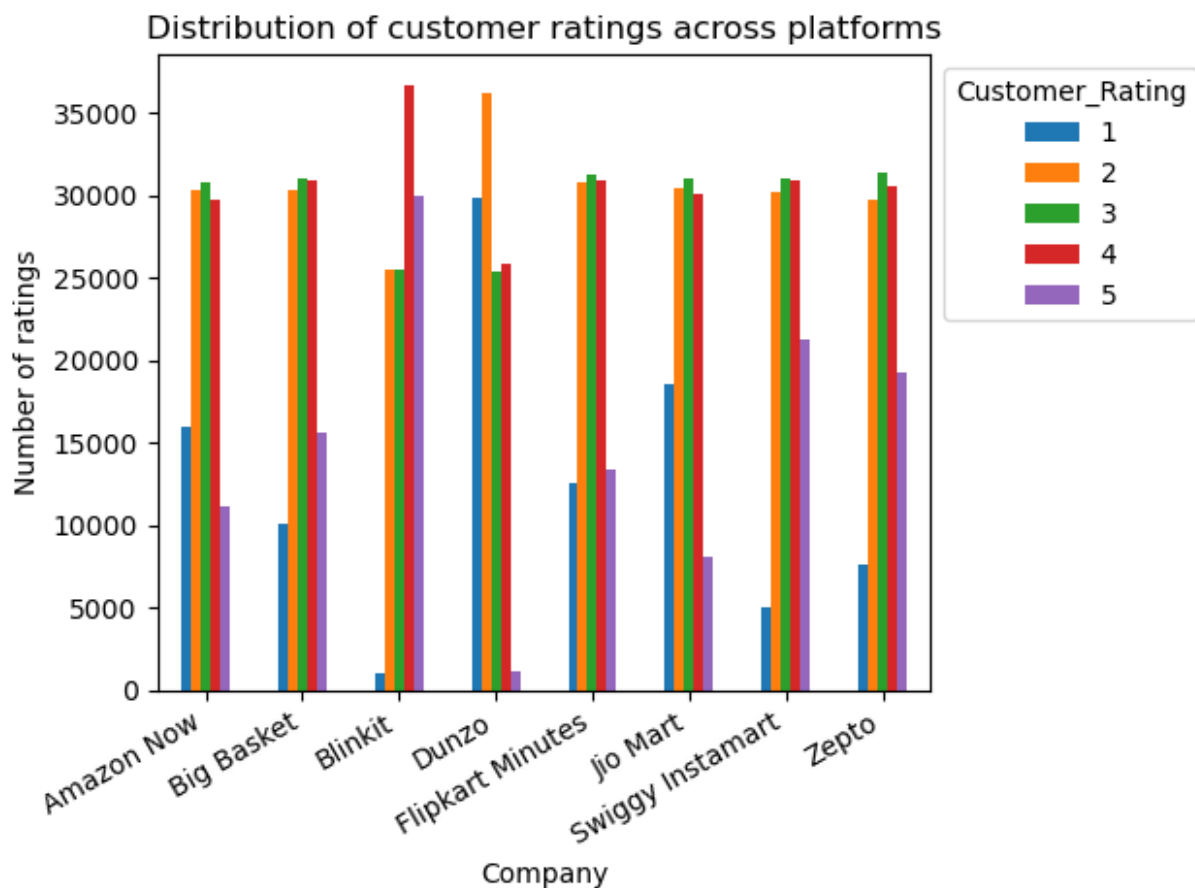
grouped_rating.plot (kind = 'bar')

plt.xlabel('Company')
plt.ylabel('Number of ratings')
plt.title('Distribution of customer ratings across platforms' )

plt.legend( title = 'Customer_Rating' , bbox_to_anchor = (1,1), loc =
'upper left')

plt.xticks (rotation = 30 , ha = 'right')

plt.tight_layout()
plt.show()
```



Q4) Does 'Delivery time' affects the 'delivery partner ratings'?

```
df.head()
```

	Order_ID	Company	City	Customer_Age	Order_Value	\
0	1000001	Swiggy Instamart	Noida	46	702	
1	1000002	Flipkart Minutes	Amritsar	56	1007	
2	1000003	Flipkart Minutes	Mumbai	18	1212	
3	1000004	Swiggy Instamart	Delhi	23	1179	
4	1000005	Dunzo	Mumbai	44	586	

	Delivery_Time_Min	Distance_Km	Items_Count	Product_Category	\
0	19	12.0	12	Dairy	
1	20	12.7	10	Snacks	
2	17	4.8	19	Personal Care	
3	6	6.4	2	Dairy	
4	12	2.4	13	Household	

	Payment_Method	Customer_Rating	Discount_Applied	\
0	Wallet	2	1	
1	Cash on Delivery	2	0	
2	Cash on Delivery	3	0	
3	Credit Card	5	1	
4	Wallet	4	0	

	Delivery_Partner_Rating
0	3
1	3
2	4
3	5
4	5

```
df['Delivery_Time_Min'].mean()
```

```
np.float64(16.506619875241626)
```

```
df['Delivery_Partner_Rating'].mean()
```

```
np.float64(3.80029480285982)
```

```
# Calculate average rating for delivery timings
```

```
x = df.groupby('Delivery_Time_Min') [
'Delivery_Partner_Rating'].mean()
x
```

```
Delivery_Time_Min
5    3.803956
6    3.798555
7    3.805901
8    3.803807
9    3.804469
10   3.801688
11   3.799953
12   3.798843
```

```
13    3.795841
14    3.797351
15    3.797958
16    3.799191
17    3.802405
18    3.801744
19    3.809601
20    3.804773
21    3.803036
22    3.804585
23    3.794215
24    3.791258
25    3.800221
26    3.795316
27    3.799794
28    3.796314
29    3.792997
30    3.788537
31    3.807610
32    3.789012
33    3.781402
34    3.770451
35    3.779251
36    3.752239
37    3.732039
38    3.741307
39    3.799097
40    3.737589
```

```
Name: Delivery_Partner_Rating, dtype: float64
```

```
# Correlation between these two columns
```

```
df['Delivery_Time_Min'].corr(df['Delivery_Partner_Rating'])
```

```
np.float64(-0.0027773408026800666)
```

```
# Line chart
```

```
plt.figure(figsize = (8,6))
```

```
x.plot(marker = '*')
```

```
plt.xlabel("Delivery Time (minutes)")
```

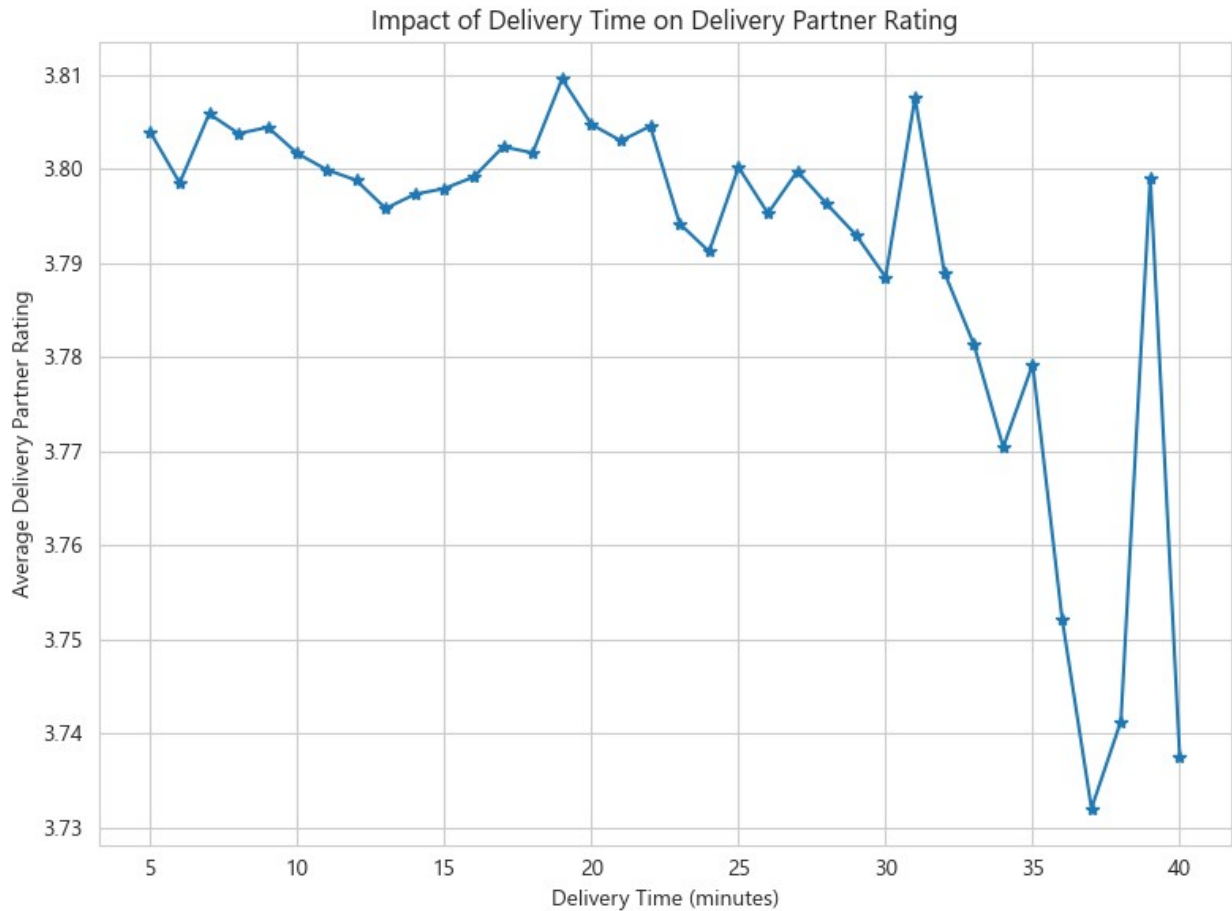
```
plt.ylabel("Average Delivery Partner Rating")
```

```
plt.title("Impact of Delivery Time on Delivery Partner Rating")
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```



Creating a new column using cut function

```
df['Delivery_Time_Bucket'] =pd.cut (
    df['Delivery_Time_Min'],
    bins = (0,10,20,30,40,50),
    labels = ('very fast delivery', 'fast
delivery', 'normal delivery', 'slow delivery', 'very slow delivery'))
```

df.head()

	Order_ID	Company	City	Customer_Age	Order_Value	\
0	1000001	Swiggy Instamart	Noida	46	702	
1	1000002	Flipkart Minutes	Amritsar	56	1007	
2	1000003	Flipkart Minutes	Mumbai	18	1212	
3	1000004	Swiggy Instamart	Delhi	23	1179	
4	1000005	Dunzo	Mumbai	44	586	

	Delivery_Time_Min	Distance_Km	Items_Count	Product_Category	\
0	19	12.0	12	Dairy	
1	20	12.7	10	Snacks	
2	17	4.8	19	Personal Care	
3	6	6.4	2	Dairy	

4	12	2.4	13	Household
	Payment_Method	Customer_Rating	Discount_Applied	\
0	Wallet	2		1
1	Cash on Delivery	2		0
2	Cash on Delivery	3		0
3	Credit Card	5		1
4	Wallet	4		0

	Delivery_Partner_Rating	Delivery_Time_Bucket
0	3	fast delivery
1	3	fast delivery
2	4	fast delivery
3	5	very fast delivery
4	5	fast delivery

```
bucket_avg = df.groupby('Delivery_Time_Bucket') [
'Delivery_Partner_Rating'].mean()
bucket_avg
```

C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\2709011709.py:1:
FutureWarning: The default of observed=False is deprecated and will be
changed to True in a future version of pandas. Pass observed=False to
retain current behavior or observed=True to adopt the future default
and silence this warning.

```
bucket_avg = df.groupby('Delivery_Time_Bucket')
[ 'Delivery_Partner_Rating'].mean()
```

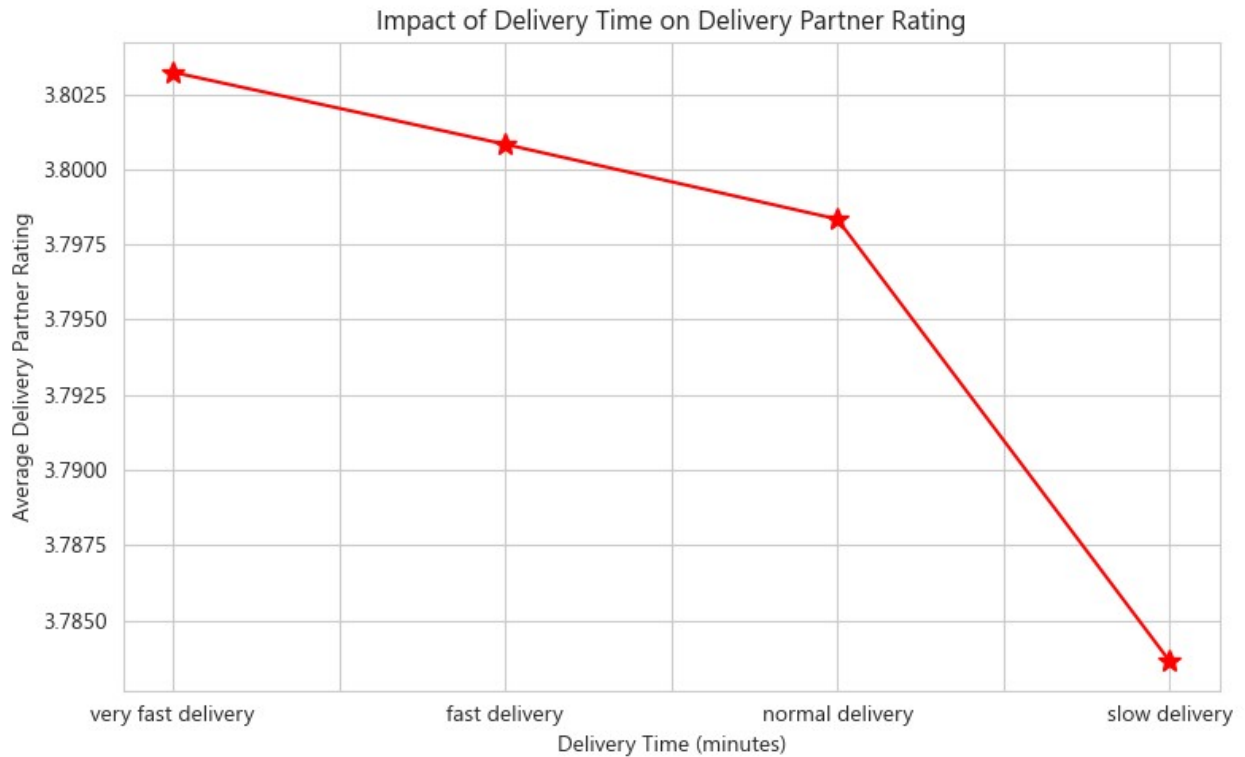
```
Delivery_Time_Bucket
very fast delivery    3.803231
fast delivery        3.800835
normal delivery      3.798345
slow delivery        3.783646
very slow delivery   NaN
Name: Delivery_Partner_Rating, dtype: float64
```

```
plt.figure(figsize = (8,5))
```

```
bucket_avg.plot (kind = 'line' , marker = '*', markersize = 10 ,
color= 'red')
```

```
plt.xlabel("Delivery Time (minutes)")
plt.ylabel("Average Delivery Partner Rating")
plt.title("Impact of Delivery Time on Delivery Partner Rating")
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```



Q5) what is the most popular Product Category on swiggy instamart, for the people of age between 30-40, in mumbai?

df.head()

	Order_ID	Company	City	Customer_Age	Order_Value	\
0	1000001	Swiggy Instamart	Noida	46	702	
1	1000002	Flipkart Minutes	Amritsar	56	1007	
2	1000003	Flipkart Minutes	Mumbai	18	1212	
3	1000004	Swiggy Instamart	Delhi	23	1179	
4	1000005	Dunzo	Mumbai	44	586	

	Delivery_Time_Min	Distance_Km	Items_Count	Product_Category	\
0	19	12.0	12	Dairy	
1	20	12.7	10	Snacks	
2	17	4.8	19	Personal Care	
3	6	6.4	2	Dairy	
4	12	2.4	13	Household	

	Payment_Method	Customer_Rating	Discount_Applied	\
0	Wallet	2	1	
1	Cash on Delivery	2	0	
2	Cash on Delivery	3	0	
3	Credit Card	5	1	

4	Wallet	4	0
---	--------	---	---

	Delivery_Partner_Rating	Delivery_Time_Bucket
0	3	fast delivery
1	3	fast delivery
2	4	fast delivery
3	5	very fast delivery
4	5	fast delivery

Filtering

```
df_PC = df [(df['Company'] == 'Swiggy Instamart' ) &
            (df['City'] == 'Mumbai') &
            ((df['Customer_Age'] >=30) & (df['Customer_Age'] <40))
            ]
```

df_PC

	Order_ID	Company	City	Customer_Age	Order_Value
\					
512	1000537	Swiggy Instamart	Mumbai	38	1157
872	1000910	Swiggy Instamart	Mumbai	32	926
1148	1001204	Swiggy Instamart	Mumbai	37	234
2783	1002919	Swiggy Instamart	Mumbai	31	423
4038	1004252	Swiggy Instamart	Mumbai	36	50
...	...	...	...	...	...
945478	1997611	Swiggy Instamart	Mumbai	37	448
947020	1999243	Swiggy Instamart	Mumbai	32	747
947036	1999259	Swiggy Instamart	Mumbai	35	50
947444	1999681	Swiggy Instamart	Mumbai	39	375
947748	1999997	Swiggy Instamart	Mumbai	33	1510

	Delivery_Time_Min	Distance_Km	Items_Count
Product_Category \			
512	14	5.7	18
Beverages			
872	20	14.2	3
Dairy			
1148	19	7.6	6
Dairy			

2783	15	4.0	10	
Dairy				
4038	15	2.2	12	
Dairy				
...	...	...	...	.
..				
945478	19	14.0	17	Fruits &
Vegetables				
947020	13	2.4	12	
Household				
947036	18	7.6	4	Personal
Care				
947444	19	8.7	6	
Household				
947748	18	10.5	9	
Beverages				

	Payment_Method	Customer_Rating	Discount_Applied	\
512	Cash on Delivery	4	1	
872	UPI	2	0	
1148	Debit Card	5	0	
2783	Cash on Delivery	2	0	
4038	Credit Card	3	0	
...	...	...	...	
945478	Cash on Delivery	5	0	
947020	UPI	5	1	
947036	Debit Card	5	0	
947444	Wallet	2	0	
947748	Debit Card	4	1	

	Delivery_Partner_Rating	Delivery_Time_Bucket
512	4	fast delivery
872	5	fast delivery
1148	4	fast delivery
2783	3	fast delivery
4038	3	fast delivery
...	...	...
945478	5	fast delivery
947020	5	fast delivery
947036	5	fast delivery
947444	4	fast delivery
947748	4	fast delivery

[2325 rows x 14 columns]

Most popular product category

y = df_PC['Product_Category'].value_counts ()

y

```
Product_Category
Dairy          368
Fruits & Vegetables  343
Groceries      341
Household      340
Snacks         329
Personal Care  305
Beverages      299
Name: count, dtype: int64
```

```
plt.figure(figsize = (8,6))
```

```
plt.xlabel("Product Category")
plt.ylabel("Number of Items")
plt.title("Product Category by Number of Items in Mumbai (Age = 30 to 40)")
```

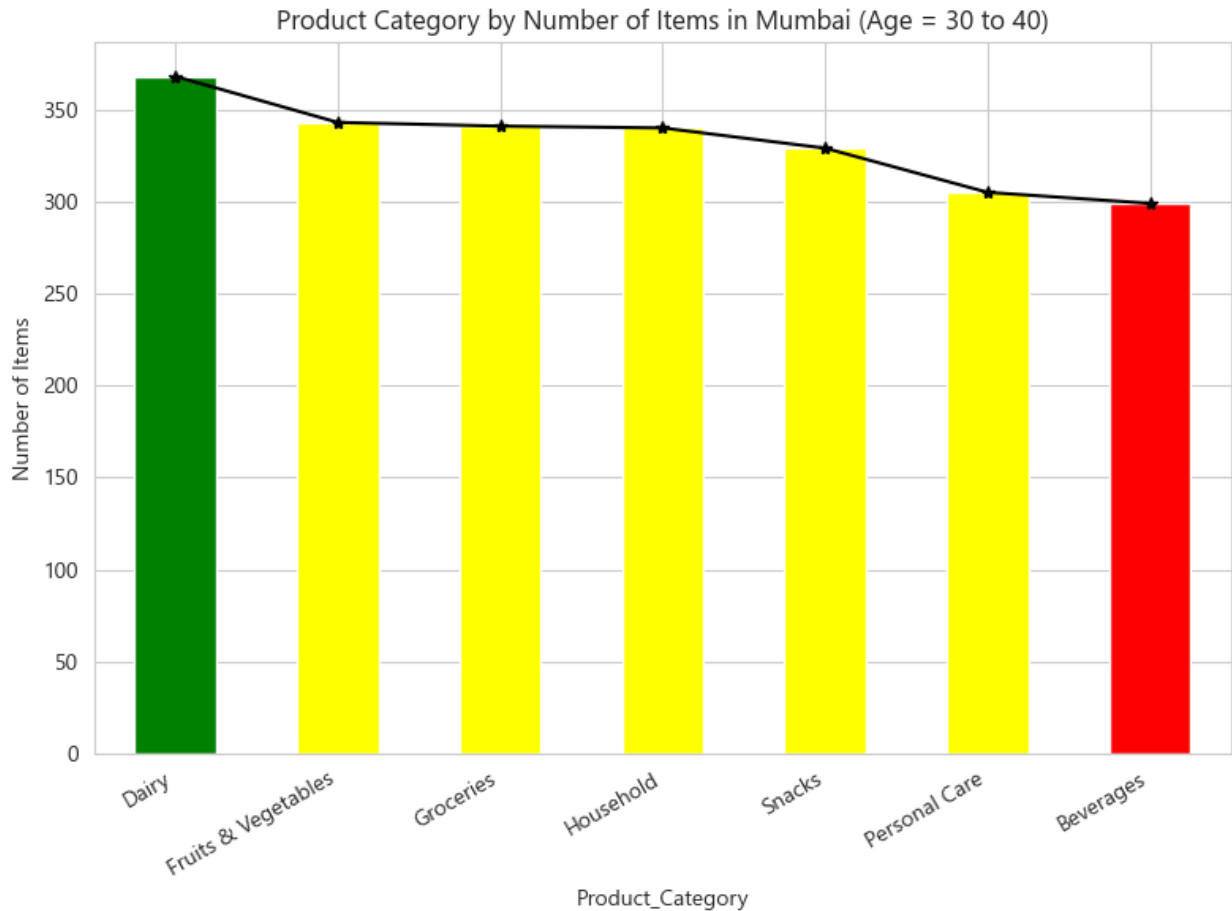
```
max_value = y.max()
min_value = y.min()
```

```
# setting colours
colors = [ 'green' if val == max_value else
           'red' if val == min_value else
           'yellow'
           for val in y]
```

```
y.plot(kind = 'bar', color= colors )
y.plot(kind = 'line', marker = '*', color = 'black')
```

```
plt.xticks(rotation = 30, ha = 'right')
```

```
plt.tight_layout()
plt.show()
```



Q6) Which cities should these company expand into based on performance?

df.head()

	Order_ID	Company	City	Customer_Age	Order_Value \
0	1000001	Swiggy Instamart	Noida	46	702
1	1000002	Flipkart Minutes	Amritsar	56	1007
2	1000003	Flipkart Minutes	Mumbai	18	1212
3	1000004	Swiggy Instamart	Delhi	23	1179
4	1000005	Dunzo	Mumbai	44	586

	Delivery_Time_Min	Distance_Km	Items_Count	Product_Category \
0	19	12.0	12	Dairy
1	20	12.7	10	Snacks
2	17	4.8	19	Personal Care
3	6	6.4	2	Dairy
4	12	2.4	13	Household

	Payment_Method	Customer_Rating	Discount_Applied \
0	Wallet	2	1

1	Cash on Delivery	2	0
2	Cash on Delivery	3	0
3	Credit Card	5	1
4	Wallet	4	0

	Delivery_Partner_Rating	Delivery_Time_Bucket
0	3	fast delivery
1	3	fast delivery
2	4	fast delivery
3	5	very fast delivery
4	5	fast delivery

Group data and apply aggregate functions

```
city_perf = df.groupby(['Company', 'City']) . agg(
    Total_Orders = ('Order_ID',
                    'count'),
    Avg_Rating =
    ('Customer_Rating', 'mean'),
    Avg_Delivery_Time =
    ('Delivery_Time_Min', 'mean'),
    Total_Revenue = ('Order_Value'
                    , 'sum')
) .reset_index()
```

city_perf

	Company	City	Total_Orders	Avg_Rating	Avg_Delivery_Time
0	Amazon Now	Amritsar	9929	2.896666	18.167388
1	Amazon Now	Bengluru	9905	3.387683	19.183342
2	Amazon Now	Chennai	9847	2.904032	18.138722
3	Amazon Now	Delhi	9918	2.989716	8.294616
4	Amazon Now	Gurgaon	9797	2.921711	18.166173
..	...	...	...	...	...
91	Zepto	Jaipur	9678	3.112213	13.145278
92	Zepto	Kolkata	9825	3.173028	8.302901
93	Zepto	Mumbai	9861	3.412230	10.152013
94	Zepto	Noida	10070	3.206653	8.297617
95	Zepto	Pune	9857	3.188495	8.352643

	Total_Revenue
0	5469439
1	5629936
2	5363559
3	5891026
4	6680388
..	...
91	4621428
92	5776955
93	6046485
94	7070769
95	5748510

[96 rows x 6 columns]

Filtering out the best cities based on given condition

```
best_cities = city_perf [
    (city_perf ['Avg_Rating'] >= 3.5) &
    (city_perf ['Avg_Delivery_Time'] <= 15)&
    (city_perf ['Total_Orders'] > city_perf
 ['Total_Orders']. median())
]
```

best_cities

	Company	City	Total_Orders	Avg_Rating	Avg_Delivery_Time	\
24	Blinkit	Amritsar	9932	3.589106	14.199154	
26	Blinkit	Chennai	9876	3.580903	14.135581	
28	Blinkit	Gurgaon	10078	3.579381	14.199345	
32	Blinkit	Kolkata	10004	3.566873	14.142743	
35	Blinkit	Pune	9908	3.583266	14.164110	
85	Zepto	Bengluru	9971	3.598636	9.198977	

	Total_Revenue
24	6006361
26	5921870
28	7374774
32	6003384
35	5930415
85	6041668

best_cities[['Company', 'City']]

	Company	City
24	Blinkit	Amritsar
26	Blinkit	Chennai
28	Blinkit	Gurgaon
32	Blinkit	Kolkata

```

35 Blinkit      Pune
85 Zepto       Bengaluru

# Bar plot using seaborn library

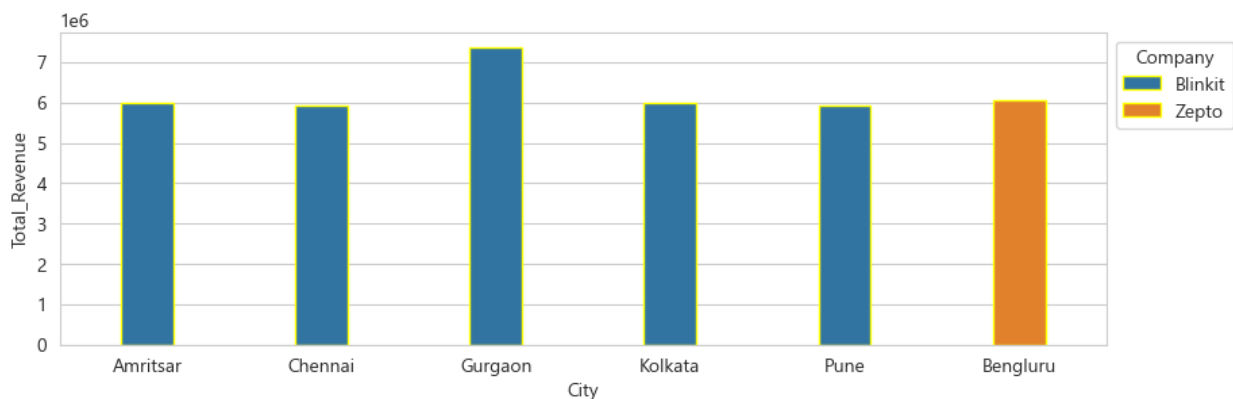
plt.figure(figsize = (10,3))

sns.barplot( data = best_cities, x= 'City', y='Total_Revenue', hue=
'Company', edgecolor = 'yellow' , width= 0.3 )

plt.legend( title = 'Company', bbox_to_anchor = (1,1) )

plt.show()

```



Q7) Are discounts increasing the order value or just reducing the revenue?

```

df.head()

```

	Order_ID	Company	City	Customer_Age	Order_Value \
0	1000001	Swiggy Instamart	Noida	46	702
1	1000002	Flipkart Minutes	Amritsar	56	1007
2	1000003	Flipkart Minutes	Mumbai	18	1212
3	1000004	Swiggy Instamart	Delhi	23	1179
4	1000005	Dunzo	Mumbai	44	586

	Delivery_Time_Min	Distance_Km	Items_Count	Product_Category \
0	19	12.0	12	Dairy
1	20	12.7	10	Snacks
2	17	4.8	19	Personal Care
3	6	6.4	2	Dairy
4	12	2.4	13	Household

	Payment_Method	Customer_Rating	Discount_Applied \
0	Wallet	2	1
1	Cash on Delivery	2	0

2	Cash on Delivery	3	0
3	Credit Card	5	1
4	Wallet	4	0

	Delivery_Partner_Rating	Delivery_Time_Bucket
0	3	fast delivery
1	3	fast delivery
2	4	fast delivery
3	5	very fast delivery
4	5	fast delivery

#Count the number of orders with & without Discount

```
df['Discount_Applied'].value_counts()
```

```
Discount_Applied
```

```
0    567906
```

```
1    379846
```

```
Name: count, dtype: int64
```

Group data - calculate average order value

```
dis_order_value = df.groupby('Discount_Applied')['Order_Value'].mean()
```

```
dis_order_value
```

```
Discount_Applied
```

```
0    476.417152
```

```
1    712.190830
```

```
Name: Order_Value, dtype: float64
```

Group data- Calculate the sum of items count

```
dis_items_count = df.groupby('Discount_Applied')['Items_Count'].sum()
```

```
dis_items_count
```

```
Discount_Applied
```

```
0    5854258
```

```
1    3920185
```

```
Name: Items_Count, dtype: int64
```

```
x = df['Discount_Applied'].unique()
```

```
x
```

```
array([1, 0])
```

```
plt.figure(figsize = (12,8))
```

#Use gradient color

```
color = sns.color_palette("coolwarm", len(x))
```

```

plt.subplot(2,2, 1)
dis_order_value.plot(kind = 'bar', width = 0.3, edgecolor = 'red' ,
color = color )
plt.title("Average Order Value")

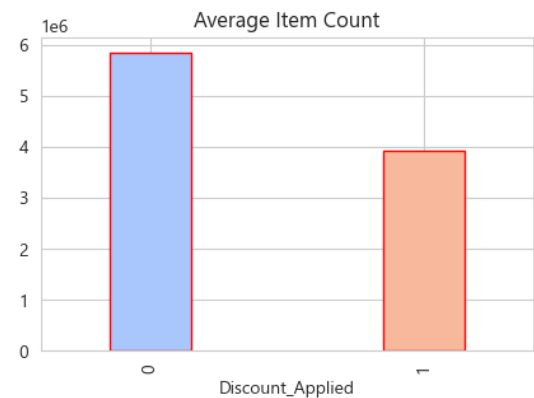
plt.subplot(2,2,2)
dis_items_count.plot(kind = 'bar', width = 0.3, edgecolor = 'r' ,
color = color )
plt.title("Average Item Count")

plt.subplots_adjust( wspace= 0.4, hspace =0.5 )

# plt.tight_layout() == Automato=ic adjust the spacing

plt.show()

```



Q8) which company has the best operational efficiency (Delivery Time vs Order Volume)?

```
df.head ()
```

	Order_ID	Company	City	Customer_Age	Order_Value \
0	1000001	Swiggy Instamart	Noida	46	702
1	1000002	Flipkart Minutes	Amritsar	56	1007
2	1000003	Flipkart Minutes	Mumbai	18	1212
3	1000004	Swiggy Instamart	Delhi	23	1179
4	1000005	Dunzo	Mumbai	44	586

	Delivery_Time_Min	Distance_Km	Items_Count	Product_Category \
0	19	12.0	12	Dairy
1	20	12.7	10	Snacks
2	17	4.8	19	Personal Care
3	6	6.4	2	Dairy
4	12	2.4	13	Household

	Payment_Method	Customer_Rating	Discount_Applied	\
0	Wallet	2	1	
1	Cash on Delivery	2	0	
2	Cash on Delivery	3	0	
3	Credit Card	5	1	
4	Wallet	4	0	

	Delivery_Partner_Rating	Delivery_Time_Bucket
0	3	fast delivery
1	3	fast delivery
2	4	fast delivery
3	5	very fast delivery
4	5	fast delivery

Group data and apply aggregate functions

```
company_efficiency = df.groupby('Company').agg(
    Total_Orders = ('Order_ID', 'count' ) ,
    Avg_Delivery_Time = ('Delivery_Time_Min' ,
'mean' )
).reset_index()
```

company_efficiency

	Company	Total_Orders	Avg_Delivery_Time
0	Amazon Now	118083	18.984155
1	Big Basket	118047	18.027345
2	Blinkit	118887	15.121771
3	Dunzo	118463	14.177060
4	Flipkart Minutes	118965	17.045080
5	Jio Mart	118267	23.013224
6	Swiggy Instamart	118475	16.072690
7	Zepto	118565	9.644313

Normalize the value for fair comparision as order value is very high as compared to delivery time

```
from sklearn.preprocessing import MinMaxScaler
```

Scaling both the column to the same range (0 to 1) so that they can be fairly compared and combined.

```
scaler = MinMaxScaler() # create a scaling object
```

```
company_efficiency[['Total_Orders_Scaled' ,
'Avg_Delivery_Time_Scaled']] = scaler.fit_transform (
    company_efficiency
[['Total_Orders' , 'Avg_Delivery_Time' ]]
```

```
)
```

company_efficiency

	Company	Total_Orders	Avg_Delivery_Time
0	Amazon Now	118083	18.984155
1	Big Basket	118047	18.027345
2	Blinkit	118887	15.121771
3	Dunzo	118463	14.177060
4	Flipkart Minutes	118965	17.045080
5	Jio Mart	118267	23.013224
6	Swiggy Instamart	118475	16.072690
7	Zepto	118565	9.644313

	Avg_Delivery_Time_Scaled
0	0.698624
1	0.627054
2	0.409716
3	0.339051
4	0.553580
5	1.000000
6	0.480845
7	0.000000

Efficiency score (hiher orders, lower time)

```
company_efficiency['Efficiency Score'] =  
company_efficiency['Total_Orders_Scaled'] -  
company_efficiency['Avg_Delivery_Time_Scaled']
```

company_efficiency

	Company	Total_Orders	Avg_Delivery_Time
0	Amazon Now	118083	18.984155
1	Big Basket	118047	18.027345
2	Blinkit	118887	15.121771
3	Dunzo	118463	14.177060

```

0.453159
4 Flipkart Minutes      118965      17.045080
1.000000
5 Jio Mart              118267      23.013224
0.239651
6 Swiggy Instamart      118475      16.072690
0.466231
7 Zepto                 118565      9.644313
0.564270

```

	Avg_Delivery_Time_Scaled	Efficiency Score
0	0.698624	-0.659408
1	0.627054	-0.627054
2	0.409716	0.505317
3	0.339051	0.114108
4	0.553580	0.446420
5	1.000000	-0.760349
6	0.480845	-0.014614
7	0.000000	0.564270

```
# Sort by best efficiency
```

```
Eff_Sort = company_efficiency.sort_values('Efficiency Score',
ascending = False)
```

```
Eff_Sort
```

	Company	Total_Orders	Avg_Delivery_Time
Total_Orders_Scaled \			
7	Zepto	118565	9.644313
0.564270			
2	Blinkit	118887	15.121771
0.915033			
4	Flipkart Minutes	118965	17.045080
1.000000			
3	Dunzo	118463	14.177060
0.453159			
6	Swiggy Instamart	118475	16.072690
0.466231			
1	Big Basket	118047	18.027345
0.000000			
0	Amazon Now	118083	18.984155
0.039216			
5	Jio Mart	118267	23.013224
0.239651			

	Avg_Delivery_Time_Scaled	Efficiency Score
7	0.000000	0.564270
2	0.409716	0.505317
4	0.553580	0.446420

3	0.339051	0.114108
6	0.480845	-0.014614
1	0.627054	-0.627054
0	0.698624	-0.659408
5	1.000000	-0.760349

```
x = company_efficiency['Efficiency Score'].unique()
```

```
x
```

```
array([-0.6594083 , -0.6270542 ,  0.5053166 ,  0.11410777,
        0.44641959,
        -0.76034858, -0.0146143 ,  0.56427015])
```

```
color = sns.color_palette("coolwarm", len(x))
```

```
plt.bar(Eff_Sort['Company'], Eff_Sort['Efficiency Score'], color =
color , edgecolor = 'black')
```

```
plt.xlabel('Company')
```

```
plt.ylabel('Efficiency Score')
```

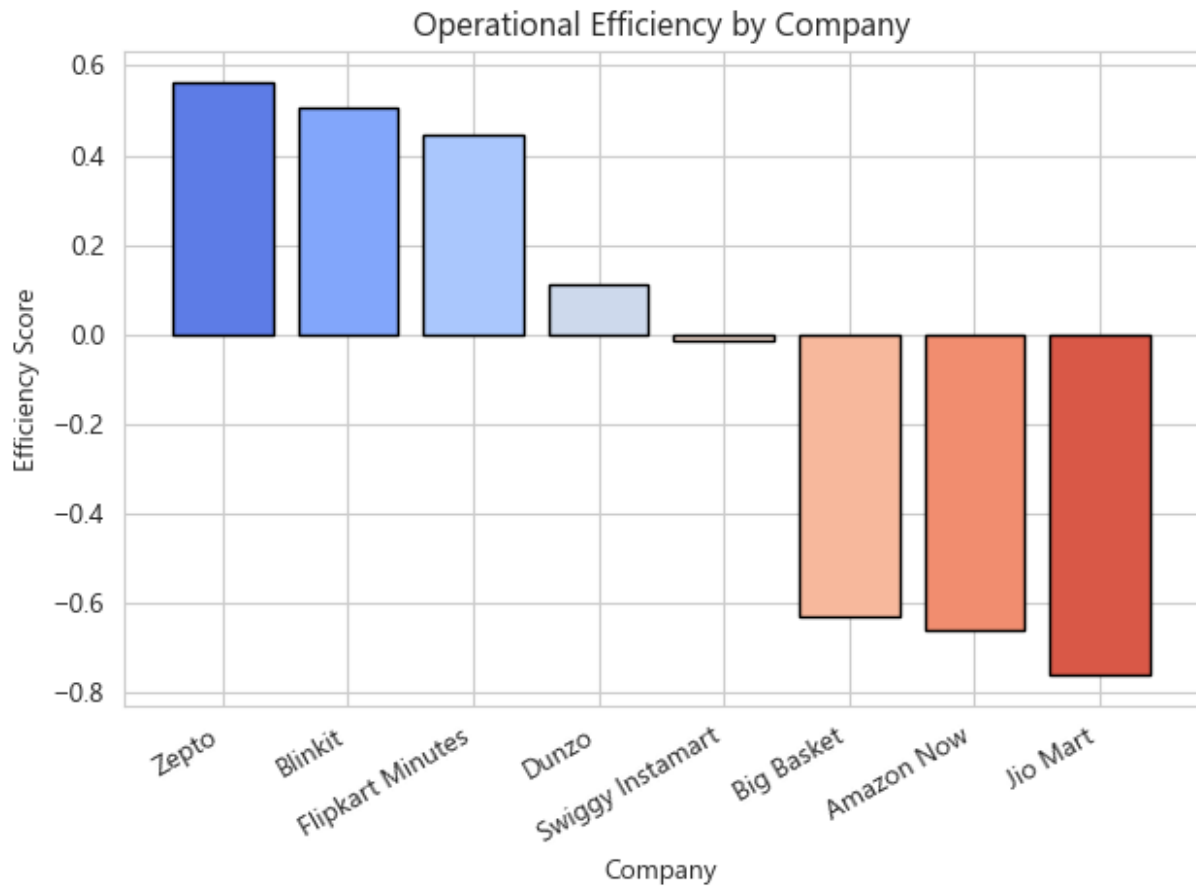
```
plt.title('Operational Efficiency by Company')
```

```
plt.grid(True)
```

```
plt.xticks(rotation = 30, ha = 'right')
```

```
plt.tight_layout()
```

```
plt.show()
```



```
# Use plotly library - Scatter Bubble plot
import plotly.express as px

# Create positive bubble size
min_eff = Eff_Sort['Efficiency Score'].min()
Eff_Sort['Bubble_Size'] = Eff_Sort['Efficiency Score'] - min_eff + 0.1

# PLOT
fig = px.scatter(Eff_Sort, x = 'Total_Orders' , y =
'Avg_Delivery_Time' , size = 'Bubble_Size', color = 'Company', text =
'Company' ,
                title = 'Operational Efficiency of Quick Commerce
Platforms',
                hover_data = {'Total_Orders' : True ,
'Avg_Delivery_Time': '::.2f' , 'Efficiency Score' : '::.3f' },
                size_max = 40)

# Improve chart
fig.update_layout (
```

```

        height = 650,
        width = 1100,
        xaxis_title = 'Total Orders (Higher is better)',
        yaxis_title = 'Average delivery timee (Lower is
better)',

        title_x = 0.5,
        template = "plotly_dark"
    )
# Make labels cleaner
fig.update_traces(textposition = 'middle center')

# Reverse Y-axis so best performance appers higher
fig.update_yaxes(autorange = 'reversed' )

fig.show()

{"config":{"plotlyServerURL":"https://plot.ly"},"data":[{"customdata":
{"bdata":"ANVjR4A04j8=","dtype":"f8","shape":"1,
1"},"hovertemplate":"Company=%{text}<br>Total_Orders=%
{x}<br>Avg_Delivery_Time=%{y:.2f}<br>Bubble_Size=%
{marker.size}<br>Efficiency
Score=%{customdata[0]:.3f}<extra></extra>","legendgroup":"Zepto","mark
er":{"color":"#636efa","size":
{"bdata":"GpIgBD3L9j8=","dtype":"f8"},"sizemode":"area","sizeref":8.90
3867102396427e-
4,"symbol":"circle"},"mode":"markers+text","name":"Zepto","orientation
":"v","showlegend":true,"text":["Zepto"],"textposition":"middle
center","type":"scatter","x":
{"bdata":"Jc8BAA==" ,"dtype":"i4"},"xaxis":"x","y":{"bdata":"0Qf/
b0NJI0A=","dtype":"f8"},"yaxis":"y"}, {"customdata":
{"bdata":"rFJRt40r4D8=","dtype":"f8","shape":"1,
1"},"hovertemplate":"Company=%{text}<br>Total_Orders=%
{x}<br>Avg_Delivery_Time=%{y:.2f}<br>Bubble_Size=%
{marker.size}<br>Efficiency
Score=%{customdata[0]:.3f}<extra></extra>","legendgroup":"Blinkit","ma
rker":{"color":"#EF553B","size":
{"bdata":"8FAXvMPZ9T8=","dtype":"f8"},"sizemode":"area","sizeref":8.90
3867102396427e-
4,"symbol":"circle"},"mode":"markers+text","name":"Blinkit","orientati
on":"v","showlegend":true,"text":["Blinkit"],"textposition":"middle
center","type":"scatter","x":
{"bdata":"Z9ABAA==" ,"dtype":"i4"},"xaxis":"x","y":{"bdata":"MN/
gxlg+LkA=","dtype":"f8"},"yaxis":"y"}, {"customdata":
{"bdata":"s09Ley0S3D8=","dtype":"f8","shape":"1,
1"},"hovertemplate":"Company=%{text}<br>Total_Orders=%
{x}<br>Avg_Delivery_Time=%{y:.2f}<br>Bubble_Size=%
{marker.size}<br>Efficiency
Score=%{customdata[0]:.3f}<extra></extra>","legendgroup":"Flipkart
Minutes","marker":{"color":"#00cc96","size":
{"bdata":"hqNBv4Xo9D8=","dtype":"f8"},"sizemode":"area","sizeref":8.90

```

3867102396427e-

```
4, "symbol": "circle", "mode": "markers+text", "name": "Flipkart  
Minutes", "orientation": "v", "showlegend": true, "text": ["Flipkart  
Minutes"], "textposition": "middle center", "type": "scatter", "x":  
{"bdata": "tdABAA==", "dtype": "i4"}, "xaxis": "x", "y":  
{"bdata": "xHEMZYoLMUA=", "dtype": "f8"}, "yaxis": "y"}, {"customdata":  
{"bdata": "MGFFwCo2vT8=", "dtype": "f8", "shape": "1,  
1"}, "hovertemplate": "Company=%{text}<br>Total_Orders=%  
{x}<br>Avg_Delivery_Time=%{y:.2f}<br>Bubble_Size=%  
{marker.size}<br>Efficiency  
Score=%{customdata[0]:.3f}<extra></extra>", "legendgroup": "Dunzo", "mark  
er": {"color": "#ab63fa", "size":  
{"bdata": "WfvGL8u7z8=", "dtype": "f8"}, "sizemode": "area", "sizeref": 8.90
```

3867102396427e-

```
4, "symbol": "circle", "mode": "markers+text", "name": "Dunzo", "orientation  
": "v", "showlegend": true, "text": ["Dunzo"], "textposition": "middle  
center", "type": "scatter", "x":  
{"bdata": "v84BAA==", "dtype": "i4"}, "xaxis": "x", "y":  
{"bdata": "laEUi6daLEA=", "dtype": "f8"}, "yaxis": "y"}, {"customdata":  
{"bdata": "gAJSARujb8=", "dtype": "f8", "shape": "1,  
1"}, "hovertemplate": "Company=%{text}<br>Total_Orders=%  
{x}<br>Avg_Delivery_Time=%{y:.2f}<br>Bubble_Size=%  
{marker.size}<br>Efficiency  
Score=%{customdata[0]:.3f}<extra></extra>", "legendgroup": "Swiggy  
Instamart", "marker": {"color": "#FFA15A", "size":  
{"bdata": "KQfYWEEQ6z8=", "dtype": "f8"}, "sizemode": "area", "sizeref": 8.90
```

3867102396427e-

```
4, "symbol": "circle", "mode": "markers+text", "name": "Swiggy  
Instamart", "orientation": "v", "showlegend": true, "text": ["Swiggy  
Instamart"], "textposition": "middle center", "type": "scatter", "x":  
{"bdata": "y84BAA==", "dtype": "i4"}, "xaxis": "x", "y":  
{"bdata": "Ges615sSMEA=", "dtype": "f8"}, "yaxis": "y"}, {"customdata":  
{"bdata": "lH6b9tMQ5L8=", "dtype": "f8", "shape": "1,  
1"}, "hovertemplate": "Company=%{text}<br>Total_Orders=%  
{x}<br>Avg_Delivery_Time=%{y:.2f}<br>Bubble_Size=%  
{marker.size}<br>Efficiency  
Score=%{customdata[0]:.3f}<extra></extra>", "legendgroup": "Big  
Basket", "marker": {"color": "#19d3f3", "size":  
{"bdata": "fUIHKZfczT8=", "dtype": "f8"}, "sizemode": "area", "sizeref": 8.90
```

3867102396427e-4, "symbol": "circle", "mode": "markers+text", "name": "Big

```
Basket", "orientation": "v", "showlegend": true, "text": ["Big  
Basket"], "textposition": "middle center", "type": "scatter", "x":  
{"bdata": "H80BAA==", "dtype": "i4"}, "xaxis": "x", "y":  
{"bdata": "6c2mFQAHMkA=", "dtype": "f8"}, "yaxis": "y"}, {"customdata":  
{"bdata": "RlNecN8Z5b8=", "dtype": "f8", "shape": "1,  
1"}, "hovertemplate": "Company=%{text}<br>Total_Orders=%  
{x}<br>Avg_Delivery_Time=%{y:.2f}<br>Bubble_Size=%  
{marker.size}<br>Efficiency  
Score=%{customdata[0]:.3f}<extra></extra>", "legendgroup": "Amazon
```

```

Now", "marker": {"color": "#FF6692", "size": {"bdata": "te/7QWm4yT8=", "dtype": "f8"}, "sizemode": "area", "sizeref": 8.903867102396427e-4, "symbol": "circle"}, "mode": "markers+text", "name": "Amazon
Now", "orientation": "v", "showlegend": true, "text": ["Amazon
Now"], "textposition": "middle center", "type": "scatter", "x": {"bdata": "Q80BAA==", "dtype": "i4"}, "xaxis": "x", "y": {"bdata": "kGqVmPH7MkA=", "dtype": "f8"}, "yaxis": "y"}, {"customdata": {"bdata": "AByqjcZU6L8=", "dtype": "f8", "shape": "1, 1"}, "hovertemplate": "Company=%{text}<br>Total_Orders=%{x}<br>Avg_Delivery_Time=%{y:.2f}<br>Bubble_Size=%{marker.size}<br>EfficiencyScore=%{customdata[0]:.3f}<extra></extra>", "legendgroup": "Jio
Mart", "marker": {"color": "#B6E880", "size": {"bdata": "mpmZmZmZuT8=", "dtype": "f8"}, "sizemode": "area", "sizeref": 8.903867102396427e-4, "symbol": "circle"}, "mode": "markers+text", "name": "Jio
Mart", "orientation": "v", "showlegend": true, "text": ["Jio
Mart"], "textposition": "middle center", "type": "scatter", "x": {"bdata": "+80BAA==", "dtype": "i4"}, "xaxis": "x", "y": {"bdata": "fi4uq2IDN0A=", "dtype": "f8"}, "yaxis": "y"}], "layout": {"height": 650, "legend": {"itemsizing": "constant", "title": {"text": "Company"}, "tracegroupgap": 0}, "template": {"data": {"bar": [{"error_x": {"color": "#f2f5fa"}, "error_y": {"color": "#f2f5fa"}, "marker": {"line": {"color": "rgb(17,17,17)", "width": 0.5}, "pattern": {"fillmode": "overlay", "size": 10, "solidity": 0.2}}, "type": "bar"}], "barpolar": [{"marker": {"line": {"color": "rgb(17,17,17)", "width": 0.5}, "pattern": {"fillmode": "overlay", "size": 10, "solidity": 0.2}}, "type": "barpolar"}], "carpet": [{"aaxis": {"endlinecolor": "#A2B1C6", "gridcolor": "#506784", "linecolor": "#506784", "minorgridcolor": "#506784", "startlinecolor": "#A2B1C6"}, "baxis": {"endlinecolor": "#A2B1C6", "gridcolor": "#506784", "linecolor": "#506784", "minorgridcolor": "#506784", "startlinecolor": "#A2B1C6"}, "type": "carpet"}], "choropleth": [{"colorbar": {"outlinewidth": 0, "ticks": ""}, "type": "choropleth"}], "contour": [{"colorbar": {"outlinewidth": 0, "ticks": ""}, "colorscale": [[0, "#0d0887"], [0.1111111111111111, "#46039f"], [0.2222222222222222, "#7201a8"], [0.3333333333333333, "#9c179e"], [0.4444444444444444, "#bd3786"], [0.5555555555555556, "#d8576b"], [0.6666666666666666, "#ed7953"], [0.7777777777777778, "#fb9f3a"], [0.8888888888888888, "#fdca26"], [1, "#f0f921"]], "type": "contour"}], "contourcarpet": [{"colorbar": {"outlinewidth": 0, "ticks": ""}, "type": "contourcarpet"}], "heatmap": [{"colorbar": {"outlinewidth": 0, "ticks": ""}, "colorscale": [[0, "#0d0887"], [0.1111111111111111, "#46039f"], [0.2222222222222222, "#7201a8"], [0.3333333333333333, "#9c179e"], [0.4444444444444444, "#bd3786"], [0.5555555555555556, "#d8576b"], [0.6666666666666666, "#ed7953"], [0.7777777777777778, "#fb9f3a"], [0.8888888888888888, "#fdca26"],

```

```

[1,"#f0f921"]], "type": "heatmap"}], "histogram": [{"marker": {"pattern":
{"fillmode": "overlay", "size": 10, "solidity": 0.2}}, "type": "histogram"}],
"histogram2d": [{"colorbar": {"linewidth": 0, "ticks": ""}, "colorscale":
[[0, "#0d0887"], [0.1111111111111111, "#46039f"],
[0.2222222222222222, "#7201a8"], [0.3333333333333333, "#9c179e"],
[0.4444444444444444, "#bd3786"], [0.5555555555555556, "#d8576b"],
[0.6666666666666666, "#ed7953"], [0.7777777777777778, "#fb9f3a"],
[0.8888888888888888, "#fdca26"],
[1, "#f0f921"]], "type": "histogram2d"}], "histogram2dcontour":
[{"colorbar": {"linewidth": 0, "ticks": ""}, "colorscale":
[[0, "#0d0887"], [0.1111111111111111, "#46039f"],
[0.2222222222222222, "#7201a8"], [0.3333333333333333, "#9c179e"],
[0.4444444444444444, "#bd3786"], [0.5555555555555556, "#d8576b"],
[0.6666666666666666, "#ed7953"], [0.7777777777777778, "#fb9f3a"],
[0.8888888888888888, "#fdca26"],
[1, "#f0f921"]], "type": "histogram2dcontour"}], "mesh3d": [{"colorbar":
{"linewidth": 0, "ticks": ""}, "type": "mesh3d"}], "parcoords": [{"line":
{"colorbar": {"linewidth": 0, "ticks": ""}, "type": "parcoords"}], "pie":
[{"automargin": true, "type": "pie"}], "scatter": [{"marker": {"line":
{"color": "#283442"}}, "type": "scatter"}], "scatter3d": [{"line":
{"colorbar": {"linewidth": 0, "ticks": ""}, "marker": {"colorbar":
{"linewidth": 0, "ticks": ""}, "type": "scatter3d"}], "scattercarpet":
[{"marker": {"colorbar":
{"linewidth": 0, "ticks": ""}, "type": "scattercarpet"}], "scattergeo":
[{"marker": {"colorbar":
{"linewidth": 0, "ticks": ""}, "type": "scattergeo"}], "scattergl":
[{"marker": {"line":
{"color": "#283442"}}, "type": "scattergl"}], "scattermap": [{"marker":
{"colorbar":
{"linewidth": 0, "ticks": ""}, "type": "scattermap"}], "scattermapbox":
[{"marker": {"colorbar":
{"linewidth": 0, "ticks": ""}, "type": "scattermapbox"}], "scatterpolar":
[{"marker": {"colorbar":
{"linewidth": 0, "ticks": ""}, "type": "scatterpolar"}], "scatterpolargl":
[{"marker": {"colorbar":
{"linewidth": 0, "ticks": ""}, "type": "scatterpolargl"}], "scatterternary":
[{"marker": {"colorbar":
{"linewidth": 0, "ticks": ""}, "type": "scatterternary"}], "surface":
[{"colorbar": {"linewidth": 0, "ticks": ""}, "colorscale":
[[0, "#0d0887"], [0.1111111111111111, "#46039f"],
[0.2222222222222222, "#7201a8"], [0.3333333333333333, "#9c179e"],
[0.4444444444444444, "#bd3786"], [0.5555555555555556, "#d8576b"],
[0.6666666666666666, "#ed7953"], [0.7777777777777778, "#fb9f3a"],
[0.8888888888888888, "#fdca26"],
[1, "#f0f921"]], "type": "surface"}], "table": [{"cells": {"fill":
{"color": "#506784"}, "line": {"color": "rgb(17,17,17)"}}, "header":
{"fill": {"color": "#2a3f5f"}, "line":
{"color": "rgb(17,17,17)"}}, "type": "table"}], "layout":
{"annotationdefaults":

```

```

{"arrowcolor": "#f2f5fa", "arrowhead": 0, "arrowwidth": 1}, "autotypenumbers": "strict", "coloraxis": {"colorbar": {"linewidth": 0, "ticks": ""}, "colorscale": {"diverging": [[0, "#8e0152"], [0.1, "#c51b7d"], [0.2, "#de77ae"], [0.3, "#f1b6da"], [0.4, "#fde0ef"], [0.5, "#f7f7f7"], [0.6, "#e6f5d0"], [0.7, "#b8e186"], [0.8, "#7fbc41"], [0.9, "#4d9221"], [1, "#276419"]], "sequential": [[0, "#0d0887"], [0.1111111111111111, "#46039f"], [0.2222222222222222, "#7201a8"], [0.3333333333333333, "#9c179e"], [0.4444444444444444, "#bd3786"], [0.5555555555555556, "#d8576b"], [0.6666666666666666, "#ed7953"], [0.7777777777777778, "#fb9f3a"], [0.8888888888888888, "#fdca26"], [1, "#f0f921"]], "sequentialminus": [[0, "#0d0887"], [0.1111111111111111, "#46039f"], [0.2222222222222222, "#7201a8"], [0.3333333333333333, "#9c179e"], [0.4444444444444444, "#bd3786"], [0.5555555555555556, "#d8576b"], [0.6666666666666666, "#ed7953"], [0.7777777777777778, "#fb9f3a"], [0.8888888888888888, "#fdca26"], [1, "#f0f921"]]}], "colorway": ["#636efa", "#EF553B", "#00cc96", "#ab63fa", "#FFA15A", "#19d3f3", "#FF6692", "#B6E880", "#FF97FF", "#FECB52"], "font": {"color": "#f2f5fa"}, "geo": {"bgcolor": "rgb(17,17,17)", "lakecolor": "rgb(17,17,17)", "landcolor": "rgb(17,17,17)", "showlakes": true, "showland": true, "subunitcolor": "#506784"}, "hoverlabel": {"align": "left"}, "hovermode": "closest", "mapbox": {"style": "dark"}, "paper_bgcolor": "rgb(17,17,17)", "plot_bgcolor": "rgb(17,17,17)", "polar": {"angularaxis": {"gridcolor": "#506784", "linecolor": "#506784", "ticks": ""}, "bgcolor": "rgb(17,17,17)", "radialaxis": {"gridcolor": "#506784", "linecolor": "#506784", "ticks": ""}}, "scene": {"xaxis": {"backgroundcolor": "rgb(17,17,17)", "gridcolor": "#506784", "gridwidth": 2, "linecolor": "#506784", "showbackground": true, "ticks": "", "zerolinecolor": "#C8D4E3"}, "yaxis": {"backgroundcolor": "rgb(17,17,17)", "gridcolor": "#506784", "gridwidth": 2, "linecolor": "#506784", "showbackground": true, "ticks": "", "zerolinecolor": "#C8D4E3"}, "zaxis": {"backgroundcolor": "rgb(17,17,17)", "gridcolor": "#506784", "gridwidth": 2, "linecolor": "#506784", "showbackground": true, "ticks": "", "zerolinecolor": "#C8D4E3"}}, "shapedefaults": {"line": {"color": "#f2f5fa"}}, "sliderdefaults": {"bgcolor": "#C8D4E3", "bordercolor": "rgb(17,17,17)", "borderwidth": 1, "tickwidth": 0}, "ternary": {"aaxis": {"gridcolor": "#506784", "linecolor": "#506784", "ticks": ""}, "baxis": {"gridcolor": "#506784", "linecolor": "#506784", "ticks": ""}, "bgcolor": "rgb(17,17,17)", "caxis": {"gridcolor": "#506784", "linecolor": "#506784", "ticks": ""}}, "title": {"x": 5.0e-2}, "updatemenue defaults": {"bgcolor": "#506784", "borderwidth": 0}, "xaxis": {"automargin": true, "gridcolor": "#283442", "linecolor": "#506784", "ticks": "", "title": {"standoff": 15}, "zerolinecolor": "#283442", "zerolinewidth": 2}, "yaxis": {"automargin": true, "gridcolor": "#283442", "linecolor": "#506784", "ticks": ""}

```

```

: "" , "title":
{"standoff":15}, {"zerolinecolor": "#283442", "zerolinewidth": 2}}, {"title":
{"text": "Operational Efficiency of Quick Commerce
Platforms", "x": 0.5}, {"width": 1100, "xaxis": {"anchor": "y", "domain":
[0,1], "title": {"text": "Total Orders (Higher is better)"}}}, {"yaxis":
{"anchor": "x", "autorange": "reversed", "domain": [0,1], "title":
{"text": "Average delivery timee (Lower is better)"}}}

```

Mini Dashboard

Creating multiple KPI (Key Performance Indicator)

```

# Create Dashboard layout
#-----

fig = plt.figure(figsize = (12,12))

sns.set_style("darkgrid") # whitegrid for white theme

fig.patch.set_facecolor('#111111') ## Make the overall figure
background dark (nothing needed for whitegrid)

#-----
# KPI calculation
#-----
total_order = df['Order_ID'].count()
total_revenue = df['Order_Value'].sum()
avg_delivery_time = df['Delivery_Time_Min'].mean()
avg_rating = df['Customer_Rating'].mean()

# -----
# KPI Cards
# -----
plt.subplot2grid((3,4) , (0,0) )
plt.axis('off')
plt.text(0.5, 0.6, f"{total_order: ,}" , fontsize = 25, ha = 'center',
color = 'white' )
plt.text(0.5, 0.4, "Total Orders", fontsize = 20 , ha = 'center',
fontstyle = 'italic', fontweight = 'bold' , color = 'white' )

plt.subplot2grid((3,4) , (0,1) )
plt.axis('off')
plt.text(0.5, 0.6, f"Rs{total_revenue: ,}" , fontsize = 25, ha =
'center' , color = 'white')
plt.text(0.5, 0.4, "Total Revenue", fontsize = 20 , ha = 'center',
fontstyle = 'italic', fontweight = 'bold' , color = 'white' )

plt.subplot2grid((3,4) , (0,2) )

```

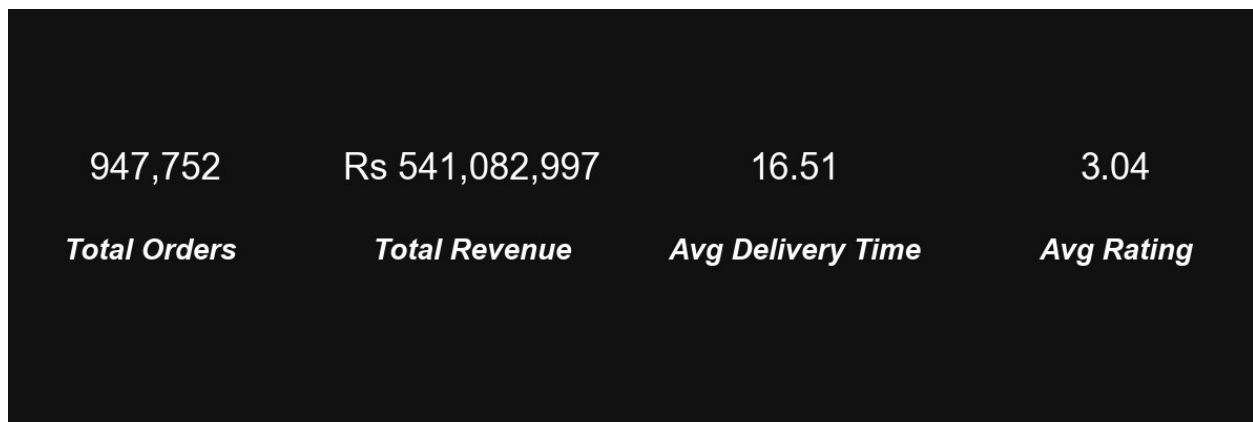
```

plt.axis('off')
plt.text(0.5, 0.6, f"{avg_delivery_time:.2f}" , fontsize = 25, ha =
'center', color = 'white' )
plt.text(0.5, 0.4, "Avg Delivery Time", fontsize = 20 , ha = 'center',
fontstyle = 'italic', fontweight = 'bold', color = 'white' )

plt.subplot2grid((3,4) , (0,3) )
plt.axis('off')
plt.text(0.5, 0.6, f"{avg_rating:.2f}" , fontsize = 25, ha = 'center',
color = 'white' )
plt.text(0.5, 0.4, "Avg Rating", fontsize = 20 , ha = 'center',
fontstyle = 'italic', fontweight = 'bold' , color = 'white' )

plt.tight_layout()
plt.show()

```



Alternative to create KPI card

```

import matplotlib.gridspec as gridspec

#-----
# Create Dashboard layout
#-----
fig = plt.figure(figsize=(14, 10))

# Use a dark-friendly style
sns.set_style("darkgrid")
fig.patch.set_facecolor('#111111') # dark background for the whole
figure

#-----
# KPI calculation

```

```

#-----
total_order = df['Order_ID'].count()
total_revenue = df['Order_Value'].sum()
avg_delivery_time = df['Delivery_Time_Min'].mean()
avg_rating = df['Customer_Rating'].mean()

#-----
# Grid setup
#-----
gs = gridspec.GridSpec(3, 4, figure=fig, hspace=0.4, wspace=0.3)

# Helper to add KPI with black card and white text
def add_kpi(ax, value_text, label_text):
    ax.set_facecolor('#000000') # pure black card
    ax.text(0.5, 0.6, str(value_text),
            fontsize=25, ha='center', va='center',
            color='white', fontweight='bold')
    ax.text(0.5, 0.35, label_text,
            fontsize=14, ha='center', va='center',
            color='white', fontstyle='italic')
    ax.axis('off')

# -----
# KPI Cards (top row)
# -----
add_kpi(fig.add_subplot(gs[0, 0]), f"{total_order:,}", "Total Orders")
add_kpi(fig.add_subplot(gs[0, 1]), f"Rs {total_revenue:,}", "Total
Revenue")
add_kpi(fig.add_subplot(gs[0, 2]), f"{avg_delivery_time:.2f}", "Avg
Delivery Time (min)")
add_kpi(fig.add_subplot(gs[0, 3]), f"{avg_rating:.2f}", "Avg Rating")

#
-----
-----
# Optional: make grid and text dark-theme friendly if you add plots
later
for ax in fig.axes:
    ax.set_facecolor('#1a1a1a') # dark panel background
    ax.tick_params(colors='white') # light tick marks/labels
    for spine in ax.spines.values():
        spine.set_color('#444444') # subtle grid/border color
    # Set default text color for labels/titles (if you add them later)
    ax.title.set_color('white')
    ax.xaxis.label.set_color('white')
    ax.yaxis.label.set_color('white')
#

```

```
plt.show()
```

947,752

Total Orders

Rs 541,082,997

Total Revenue

16.51

Avg Delivery Time (min)

3.04

Avg Rating

Creating KPI Charts

```
#-----  
# Company level metrices  
# -----  
  
company_metrics = df.groupby('Company').agg (  
    Orders = ('Order_ID', 'count'),  
    Revenue = ('Order_Value' , 'sum' ),  
    Avg_Delivery_Time = ('Delivery_Time_Min',  
    'mean'),  
    Avg_Rating = ('Customer_Rating' , 'mean')  
    ).reset_index()  
  
#-----  
# Charts Section  
  
plt.figure(figsize=(18, 12))  
  
# 1) Orders by company  
  
plt.subplot2grid((2,2) , (0,0))  
sns.barplot (data = company_metrics, x = 'Company', y = 'Orders',  
palette = 'Set2', errorbar = None, hue = 'Company', legend = False )  
plt.title ( "Orders by Company" , fontsize= 20 )  
plt.xticks(rotation = 30, ha = 'right')  
  
# 2) Revenue by company  
  
plt.subplot2grid((2,2) , (0,1))  
sns.barplot (data = company_metrics, x = 'Company', y = 'Revenue',  
palette = 'Set3', errorbar = None, hue = 'Company', legend = False )
```

```

plt.title ( "Revenue by Company" , fontsize= 20 )
plt.xticks(rotation = 30, ha = 'right')

# 3) Avg Delivery Time by Company

plt.subplot2grid((2,2) , (1,0))
sns.barplot (data = company_metrics, x = 'Company', y =
'Avg_Delivery_Time', palette = 'tab10', errorbar = None, hue =
'Company', legend = False )
plt.title ( "Avg_Delivery_Time by Company" , fontsize= 20 )
plt.xticks(rotation = 30, ha = 'right')

# 4) Avg Rating by Company

plt.subplot2grid((2,2) , (1,1))
sns.barplot (data = company_metrics, x = 'Company', y = 'Avg_Rating',
palette = 'bright', errorbar = None, hue = 'Company', legend = False )
plt.title ( "Avg_Rating by Company" , fontsize= 20 )
plt.xticks(rotation = 30, ha = 'right')

# -----
# Dashboard Title
# -----

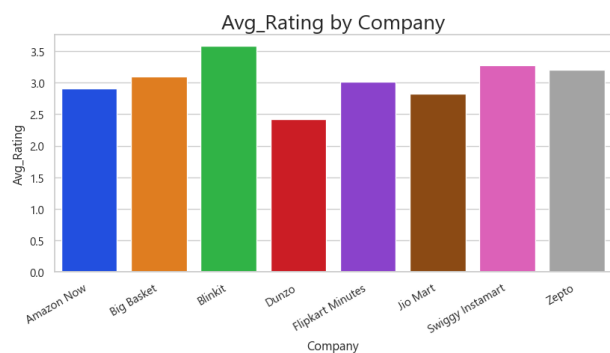
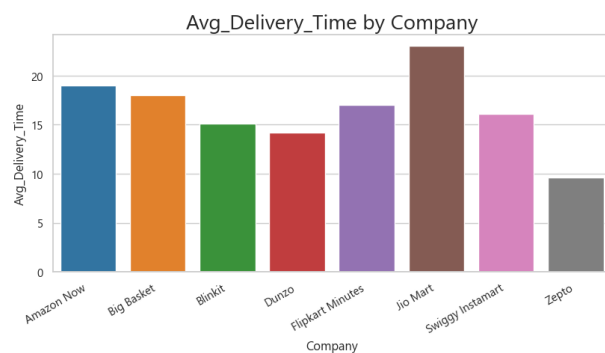
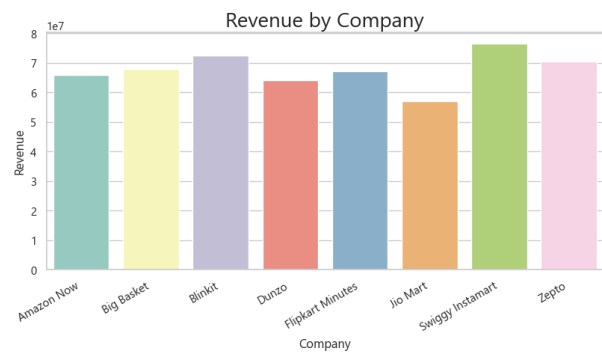
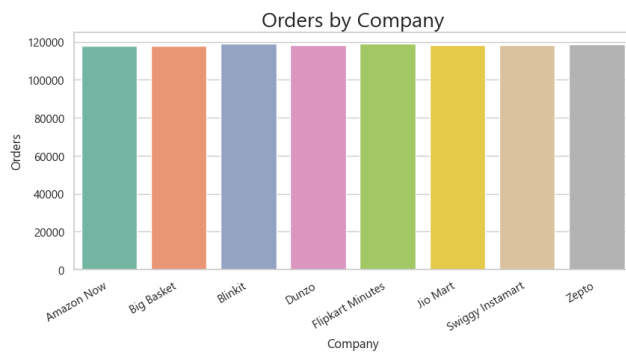
plt.rcParams ['font.family'] = "Segoe UI Emoji"

plt.suptitle(" 📊 Quick Commerce KPI Dashboard 📊 " , fontsize = 24,
color ='navy', ha = 'center', fontweight = 'bold' )

plt.subplots_adjust(
    left=0.08,      # Left-side margin
    right=0.95,     # Right-side margin
    bottom=0.12,    # Bottom margin
    top=0.82,       # Space reserved for dashboard title
    wspace=0.1,     # Horizontal space between left and right charts
    hspace=0.65     # Vertical space between top and bottom charts
)

plt.show ()

```



Alternative KPI Chart creation

```
# =====  
# QUICK COMMERCE KPI DASHBOARD  
# =====  
  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from matplotlib.ticker import FuncFormatter  
  
# -----  
# 1. CREATE COMPANY-LEVEL KPI TABLE  
# -----  
  
company_metrics = (  
    df.groupby("Company")  
        .agg(  
            Orders=("Order_ID", "count"),  
            Revenue=("Order_Value", "sum"),  
            Avg_Delivery_Time=("Delivery_Time_Min", "mean"),  
            Avg_Rating=("Customer_Rating", "mean")  
        )  
    .reset_index()
```

```

)

# Round decimal values
company_metrics["Avg_Delivery_Time"] = (
    company_metrics["Avg_Delivery_Time"].round(1)
)

company_metrics["Avg_Rating"] = (
    company_metrics["Avg_Rating"].round(2)
)

# -----
# 2. DASHBOARD STYLE
# -----

sns.set_theme(style="whitegrid")

plt.rcParams["font.family"] = "sans-serif"
plt.rcParams["font.size"] = 11

fig, axes = plt.subplots(
    nrows=2,
    ncols=2,
    figsize=(18, 12)
)

# Overall dashboard background
fig.patch.set_facecolor("#F4F6F7")

# -----
# 3. COMMON CHART FORMATTING FUNCTION
# -----

def style_chart(ax, title, xlabel):
    ax.set_title(
        title,
        fontsize=16,
        fontweight="bold",
        color="#1B2631",
        pad=15
    )

    ax.set_xlabel(
        xlabel,
        fontsize=11,
        fontweight="bold"
    )

```

```

ax.set_ylabel("")

# Keep only horizontal value-axis gridlines
ax.grid(axis="x", linestyle="--", alpha=0.30)
ax.grid(axis="y", visible=False)

# Remove extra chart borders
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.spines["left"].set_visible(False)

# Individual chart card background
ax.set_facecolor("white")

# -----
# 4. ORDERS BY COMPANY
# -----

orders_data = company_metrics.sort_values(
    by="Orders",
    ascending=True
)

ax1 = axes[0, 0]

sns.barplot(
    data=orders_data,
    x="Orders",
    y="Company",
    palette="Blues_d",
    ci=None,
    ax=ax1
)

style_chart(
    ax1,
    "Orders by Company",
    "Number of Orders"
)

# Add labels manually
orders_max = orders_data["Orders"].max()

for bar in ax1.patches:
    value = bar.get_width()

    ax1.text(
        value + (orders_max * 0.015),
        bar.get_y() + bar.get_height() / 2,

```

```

        f"{value:,.0f}",
        va="center",
        fontsize=10,
        fontweight="bold",
        color="#1B2631"
    )

# Add space at the end for labels
ax1.set_xlim(0, orders_max * 1.18)

# -----
# 5. REVENUE BY COMPANY
# -----

revenue_data = company_metrics.sort_values(
    by="Revenue",
    ascending=True
)

ax2 = axes[0, 1]

sns.barplot(
    data=revenue_data,
    x="Revenue",
    y="Company",
    palette="YlOrBr",
    ci=None,
    ax=ax2
)

style_chart(
    ax2,
    "Revenue by Company",
    "Revenue"
)

# Revenue axis format
ax2.xaxis.set_major_formatter(
    FuncFormatter(lambda x, pos: f"{x / 1000000:.1f}M")
)

# Add revenue labels manually
revenue_max = revenue_data["Revenue"].max()

for bar in ax2.patches:
    value = bar.get_width()

    ax2.text(
        value + (revenue_max * 0.015),

```

```

        bar.get_y() + bar.get_height() / 2,
        f"{value / 1000000:.2f}M",
        va="center",
        fontsize=10,
        fontweight="bold",
        color="#1B2631"
    )

ax2.set_xlim(0, revenue_max * 1.22)

# -----
# 6. AVERAGE DELIVERY TIME BY COMPANY
# -----

delivery_data = company_metrics.sort_values(
    by="Avg_Delivery_Time",
    ascending=True
)

ax3 = axes[1, 0]

sns.barplot(
    data=delivery_data,
    x="Avg_Delivery_Time",
    y="Company",
    palette="magma",
    ci=None,
    ax=ax3
)

style_chart(
    ax3,
    "Average Delivery Time by Company",
    "Delivery Time (Minutes)"
)

# Add delivery-time labels manually
delivery_max = delivery_data["Avg_Delivery_Time"].max()

for bar in ax3.patches:
    value = bar.get_width()

    ax3.text(
        value + (delivery_max * 0.02),
        bar.get_y() + bar.get_height() / 2,
        f"{value:.1f} min",
        va="center",
        fontsize=10,
        fontweight="bold",
    )

```

```

        color="#1B2631"
    )
ax3.set_xlim(0, delivery_max * 1.25)

# -----
# 7. AVERAGE CUSTOMER RATING BY COMPANY
# -----

rating_data = company_metrics.sort_values(
    by="Avg_Rating",
    ascending=True
)

ax4 = axes[1, 1]

sns.barplot(
    data=rating_data,
    x="Avg_Rating",
    y="Company",
    palette="viridis",
    ci=None,
    ax=ax4
)

style_chart(
    ax4,
    "Average Customer Rating by Company",
    "Customer Rating (Out of 5)"
)

# Ratings are usually between 0 and 5
ax4.set_xlim(0, 5.8)

# Add rating labels manually
for bar in ax4.patches:
    value = bar.get_width()

    ax4.text(
        value + 0.06,
        bar.get_y() + bar.get_height() / 2,
        f"{value:.2f}",
        va="center",
        fontsize=10,
        fontweight="bold",
        color="#1B2631"
    )

```

```

# -----
# 8. MAIN DASHBOARD TITLE
# -----

fig.suptitle(
    "Quick Commerce KPI Dashboard",
    fontsize=25,
    fontweight="bold",
    color="#0B1F5E",
    y=0.98
)

fig.text(
    0.5,
    0.945,
    "Company-Level Performance Overview",
    ha="center",
    fontsize=13,
    color="#566573"
)

# Adjust spacing so titles and labels never overlap
plt.tight_layout(rect=[0, 0, 1, 0.90])

plt.show()

```

C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\2724559581.py:99:
FutureWarning:

The `ci` parameter is deprecated. Use `errorbar=None` for the same effect.

C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\2724559581.py:99:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\2724559581.py:145:
FutureWarning:

The `ci` parameter is deprecated. Use `errorbar=None` for the same

effect.

C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\2724559581.py:145:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\2724559581.py:195:
FutureWarning:

The `ci` parameter is deprecated. Use `errorbar=None` for the same effect.

C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\2724559581.py:195:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\2724559581.py:240:
FutureWarning:

The `ci` parameter is deprecated. Use `errorbar=None` for the same effect.

C:\Users\ao808\AppData\Local\Temp\ipykernel_11060\2724559581.py:240:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

Quick Commerce KPI Dashboard

Company-Level Performance Overview

